

01236:学校网络

描述

若干学校连接到一个计算机网络。这些学校之间达成了协议：每所学校维护一个软件分发学校列表（“接收学校”）。请注意你需要编写一个程序，计算出为了使新软件能够按照协议覆盖网络中的所有学校，必须接收新软件副本的最少学校数量

输入

第一行包含一个整数 N ：网络中学校数量 ($2 \leq N \leq 100$)。学校用前 N 个正整数标识。接下来的 N 行分别描述

你的程序应该向标准输出写入两行。第一行应该包含一个正整数：子任务 A 的解。第二行应该包含子任务 B 的解。

示例输入

```
5
2 4 3 0
4 5 0
0
0
1 0
```

示例输出

```
1
2
```

```

n = int(input())
from collections import defaultdict
adj = {}
for i in range(n):
    mylist = list(map(int, input().split()))[:-1]
    adj[i] = [mylist[i]-1 for i in range(len(mylist))]
# 1. 正向 DFS, 记录完成顺序
visited = [False] * n
stack = []
def dfs1(u):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs1(v)
    stack.append(u) # 回溯时压入栈
for i in range(n):
    if not visited[i]:
        dfs1(i)
# 2. 创建反向图
rev_adj = [[] for _ in range(n)]
for u in range(n):
    for v in adj[u]:
        rev_adj[v].append(u)
# 3. 反向 DFS, 提取 SCC
visited = [False] * n
sccs = []
def dfs2(u, current_scc):
    visited[u] = True
    current_scc.append(u)
    for v in rev_adj[u]:
        if not visited[v]:
            dfs2(v, current_scc)
while stack:
    u = stack.pop()
    if not visited[u]:
        current_scc = []
        dfs2(u, current_scc)
        sccs.append(current_scc)
# 给每个点分配SCC编号
scc_id = [0] * n
for i in range(len(sccs)):
    for node in sccs[i]:
        scc_id[node] = i

```

```

# 构建缩点后的 DAG & 计算出度
dag = [[] for _ in range(len(sccs))]
in_degree = [0] * len(sccs)
out_degree = [0] * len(sccs)
edges = set() # 防止重复建边
for u in range(n):
    for v in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a != b and (a, b) not in edges:
            edges.add((a, b))
            dag[a].append(b)
            in_degree[b] += 1
            out_degree[a] += 1
print(in_degree.count(0))
if len(sccs) == 1:
    print(0)
else:
    print(max(in_degree.count(0), out_degree.count(0)))

```

P2656 采蘑菇

题目描述

小胖和 ZYR 要去 ESQMS 森林采蘑菇。

ESQMS 森林间有 N 个小树丛， M 条小径，每条小径都是单向的，连接两个小树丛，上面都有一定数量的蘑菇。小胖和 ZYR 经过某条小径一次，可以采走这条路上所有的蘑菇。由于 ESQMS 森林是一片神奇的沃土，所以一条路上的蘑菇被采过后，又会长出一些新的蘑菇，数量为原来蘑菇的数量乘上这条路的“恢复系数”，再下取整。

比如，一条路上有 4 个蘑菇，这条路的“恢复系数”为 0.7，则第一~四次经过这条路径所能采到的蘑菇数量分别为 4, 2, 1, 0。

现在，小胖和 ZYR 从 S 号小树丛出发，求他们最多能采到多少蘑菇。

输入格式

第一行两个整数， N 和 M 。

第二行到第 $M + 1$ 行，每行四个数，分别表示一条小路的起点，终点，初始蘑菇数，恢复系数。

第 $M + 2$ 行，一个整数 S 。

输出格式

一行一个整数，表示最多能采到多少蘑菇，保证答案不超过 $(2^{31} - 1)$ 。

输入输出样例 #1

输入 #1

3 3
1 2 4 0.5
1 3 7 0.1
2 3 4 0.6
1

输出 #1

8

```

import sys
sys.setrecursionlimit(10000000)
from collections import deque
data = sys.stdin.read().split()
index = 0
n,m = int(data[index]),int(data[index+1])
index += 2
adj = [[] for _ in range(n)]
for _ in range(m):
    a,b,c,d = int(data[index]),int(data[index+1]),int(data[index+2]),float(data[index+3])
    index += 4
    a -= 1
    b -= 1
    adj[a].append((b,c,d))
def tarjan_scc(n, adj):
    dfn = [-1] * n      # 搜索次序
    low = [-1] * n      # 最低链接值
    stack = []           # 辅助栈
    in_stack = [False] * n
    timer = 0
    sccs = []            # 存储最终结果
    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        stack.append(u)
        in_stack[u] = True
        for v,w,s in adj[u]:
            if dfn[v] == -1: # 情况 A: 邻居未访问
                dfs(v)
                low[u] = min(low[u], low[v])
            elif in_stack[v]: # 情况 B: 邻居在栈中 (回边)
                low[u] = min(low[u], dfn[v])
        # 判定强连通分量的根
        if low[u] == dfn[u]:
            current_scc = []
            while True:
                node = stack.pop()
                in_stack[node] = False
                current_scc.append(node)
                if node == u:
                    break
            sccs.append(current_scc)

```

```

    for i in range(n):
        if dfn[i] == -1:
            dfs(i)
    return sccs
sccs = tarjan_scc(n,adj)
scc_id = [0] * n
cnt = len(sccs)
scc_mr = [0]*cnt
for i in range(len(sccs)):
    for node in sccs[i]:
        scc_id[node] = i
# 构建缩点后的 DAG & 计算出度
dag = [[] for _ in range(len(sccs))]
graph = [[0]*len(sccs) for _ in range(len(sccs))]
indegree = [0] * len(sccs)
edges = set()
for u in range(n):
    for v,w,s in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a == b:
            t = 0
            while w:
                t += w
                w = int(w * s + 1e-8)
            scc_mr[a] += t
        if a != b :
            graph[a][b] = max(graph[a][b],w)
            if (a,b) not in edges:
                edges.add((a,b))
                dag[a].append(b)
                indegree[b] += 1

q = deque()
deg = indegree[:]
for i in range(len(sccs)):
    if deg[i] == 0:
        q.append(i)
topo = []
while q:
    u = q.popleft()
    topo.append(u)
    for v in dag[u]:
        deg[v] -= 1

```

```

        if deg[v] == 0:
            q.append(v)
start = scc_id[int(data[index])-1]
reach1 = [False] * len(sccs)
def dfs1(u):
    reach1[u] = True
    for v in dag[u]:
        if not reach1[v]:
            dfs1(v)
dfs1(start)
INF = -10**18
mn = [INF] * cnt
mn[start] = scc_mr[start]
for u in topo:
    if mn[u] == INF:
        continue
    if not reach1[u]:
        continue
    for v in dag[u]:
        if not reach1[v]:
            continue
        mn[v] = max(
            mn[v],
            scc_mr[v]+mn[u]+graph[u][v])
ans = 0
for i in range(cnt):
    if reach1[i] and mn[i] != INF:
        ans = max(
            ans,
            mn[i]
        )
print(ans)

```

P1073 [NOIP 2009 提高组] 最优贸易

题目描述

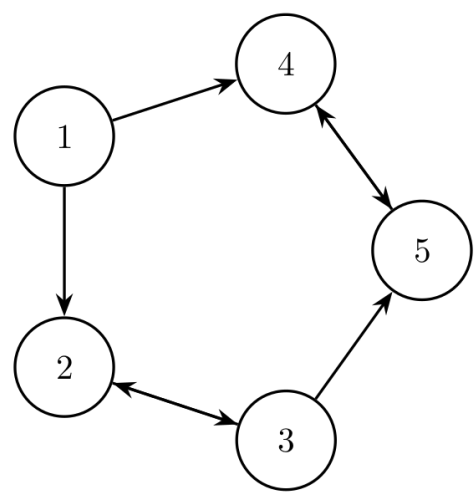
C 国有 n 个大城市 and m 条道路，每条道路连接这 n 个城市中的某两个城市。任意两个城市之间最多只有一条道路直接相连。这 m 条道路中有一部分为单向通行的道路，一部分为双向通行的道路，双向通行的道路在统计条数时也计为 1 条。

C 国幅员辽阔，各地的资源分布情况各不相同，这就导致了同一种商品在不同城市的价格不一定相同。但是，同一种商品在同一个城市的买入价和卖出价始终是相同的。

商人阿龙来到 C 国旅游。当他得知同一种商品在不同城市的价格可能会不同这一信息之后，便决定在旅

游的同时，利用商品在不同城市中的差价赚回一点旅费。设 C 国 n 个城市的标号从 $1 \sim n$ ，阿龙决定从 1 号城市出发，并最终在 n 号城市结束自己的旅行。在旅游的过程中，任何城市可以重复经过多次，但不要求经过所有 n 个城市。阿龙通过这样的贸易方式赚取旅费：他会选择一个经过的城市买入他最喜欢的商品——水晶球，并在之后经过的另一个城市卖出这个水晶球，用赚取的差价当做旅费。由于阿龙主要是来 C 国旅游，他决定这个贸易只进行最多一次，当然，在赚不到差价的情况下他就无需进行贸易。

假设 C 国有 5 个大城市，城市的编号和道路连接情况如下图，单向箭头表示这条道路为单向通行，双向箭头表示这条道路为双向通行。



假设 $1 \sim n$ 号城市的水晶球价格分别为 4, 3, 5, 6, 1。
阿龙可以选择如下一条线路： $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$ ，并在 2 号城市以 3 的价格买入水晶球，在 3 号城市以 5 的价格卖出水晶球，赚取的旅费数为 2。
阿龙也可以选择如下一条线路： $1 \rightarrow 4 \rightarrow 5 \rightarrow 4 \rightarrow 5$ ，并在第 1 次到达 5 号城市时以 1 的价格买入水晶球，在第 2 次到达 4 号城市时以 6 的价格卖出水晶球，赚取的旅费数为 5。
现在给出 n 个城市的水晶球价格， m 条道路的信息（每条道路所连接的两个城市的编号以及该条道路的通行情况）。请你告诉阿龙，他最多能赚取多少旅费。

输入格式

第一行包含 2 个正整数 n 和 m ，中间用一个空格隔开，分别表示城市的数目和道路的数目。
第二行 n 个正整数，每两个整数之间用一个空格隔开，按标号顺序分别表示这 n 个城市的商品价格。
接下来 m 行，每行有 3 个正整数 x, y, z ，每两个整数之间用一个空格隔开。如果 $z = 1$ ，表示这条道路是城市 x 到城市 y 的单向道路；如果 $z = 2$ ，表示这条道路为城市 x 和城市 y 之间的双向道路。

输出格式

一个整数，表示最多能赚取的旅费。如果没有进行贸易，则输出 0。

输入输出样例 #1

输入 #1

5 5
4 3 5 6 1
1 2 1
1 4 1
2 3 2
3 5 1
4 5 2

输出 #1

5

```

import sys
from collections import deque
data = sys.stdin.read().split()
n,m = int(data[0]),int(data[1])
index = 2
costs = [int(data[index+j]) for j in range(n)]
index += n
adj = [[] for _ in range(n)]
for _ in range(m):
    a,b,c = int(data[index]),int(data[index+1]),int(data[index+2])
    index += 3
    a -= 1
    b -= 1
    if c == 1:
        adj[a].append(b)
    else:
        adj[a].append(b)
        adj[b].append(a)
def kosaraju(n, adj):
    # 1. 正向 DFS, 记录完成顺序
    visited = [False] * n
    stack = []
    def dfs1(u):
        visited[u] = True
        for v in adj[u]:
            if not visited[v]:
                dfs1(v)
        stack.append(u) # 回溯时压入栈
    for i in range(n):
        if not visited[i]:
            dfs1(i)
    # 2. 创建反向图
    rev_adj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            rev_adj[v].append(u)
    # 3. 反向 DFS, 提取 SCC
    visited = [False] * n
    sccs = []
    def dfs2(u, current_scc):
        visited[u] = True
        current_scc.append(u)
        for v in rev_adj[u]:

```

```

        if not visited[v]:
            dfs2(v, current_scc)
    while stack:
        u = stack.pop()
        if not visited[u]:
            current_scc = []
            dfs2(u, current_scc)
            sccs.append(current_scc)
    return sccs
sccs = kosaraju(n,adj)
scc_id = [0] * n
scc_min = [float("inf")]*(len(sccs))
scc_max = [-float("inf")]*(len(sccs))
for i in range(len(sccs)):
    for node in sccs[i]:
        scc_id[node] = i
        scc_min[i] = min(scc_min[i],costs[node])
        scc_max[i] = max(scc_max[i],costs[node])
# 构建缩点后的 DAG & 计算出度
dag = [[] for _ in range(len(sccs))]
rdag = [[] for _ in range(len(sccs))]
indegree = [0]*len(sccs)
edges = set() # 防止重复建边
for u in range(n):
    for v in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a != b and (a, b) not in edges:
            edges.add((a, b))
            dag[a].append(b)
            rdag[b].append(a)
            indegree[b] += 1
q = deque()
deg = indegree[:]
for i in range(len(sccs)):
    if deg[i] == 0:
        q.append(i)
topo = []
while q:
    u = q.popleft()
    topo.append(u)
    for v in dag[u]:
        deg[v] -= 1

```

```

        if deg[v] == 0:
            q.append(v)
start = scc_id[0]
end = scc_id[n-1]
reach1 = [False] * len(sccs)
def dfs1(u):
    reach1[u] = True
    for v in dag[u]:
        if not reach1[v]:
            dfs1(v)
dfs1(start)
reach2 = [False] * len(sccs)
def dfs2(u):
    reach2[u] = True
    for v in rdag[u]:
        if not reach2[v]:
            dfs2(v)
dfs2(end)
q = deque()
deg = indegree[:]
for i in range(len(sccs)):
    if deg[i] == 0:
        q.append(i)
topo = []
while q:
    u = q.popleft()
    topo.append(u)
    for v in dag[u]:
        deg[v] -= 1
        if deg[v] == 0:
            q.append(v)
INF = 10**18
cnt = len(sccs)
mn = [INF] * cnt
mn[start] = scc_min[start]
for u in topo:
    if mn[u] == INF:
        continue
    if not (reach1[u] and reach2[u]):
        continue
    for v in dag[u]:
        if not (reach1[v] and reach2[v]):
            continue

```

```

        mn[v] = min(
            mn[v],
            min(mn[u], scc_min[v])
        )
ans = 0
for i in range(cnt):
    if reach1[i] and reach2[i] and mn[i] != INF:
        ans = max(
            ans,
            scc_max[i] - mn[i]
        )
print(ans)

```

P4742 [Wind Festival] Running In The Sky

题目背景

[Night – 20 : 02 P.M.]

夜空真美啊……但是……快要结束了呢……

题目描述

一天的活动过后，所有学生都停下来欣赏夜空下点亮的风筝。*Curtis Nishikino*想要以更近的视角感受一下，所以她跑到空中的风筝上去了(这对于一个妹子来说有点匪夷所思)！每只风筝上的灯光都有一个亮度 k_i 。由于风的作用，一些风筝缠在了一起。但是这并不会破坏美妙的气氛，缠在一起的风筝会将灯光汇聚起来，形成更亮的光源！

*Curtis Nishikino*已经知道了一些风筝间的关系，比如给出一对风筝 (a, b) ，这意味着她可以从 a 跑到 b 上去，但是不能返回。

现在，请帮她找到一条路径(她可以到达一只风筝多次，但只在第一次到达时她会去感受上面的灯光)，使得她可以感受到最多的光亮。同时请告诉她这条路径上单只风筝的最大亮度，如果有多条符合条件的路径，输出能产生最大单只风筝亮度的答案。

输入格式

第一行两个整数 n 和 m 。 n 是风筝的数量， m 是风筝间关系对的数量。

接下来一行 n 个整数 k_i 。

接下来 m 行，每行两个整数 a 和 b ，即 *Curtis* 可以从 a 跑到 b 。

输出格式

一行两个整数。*Curtis* 在计算出的路径上感受到的亮度和，这条路径上的单只风筝最大亮度。

输入输出样例 #1

输入 #1

5 5
8 9 11 6 7
1 2
2 3
2 4
4 5
5 2

输出 #1

41 11

```

import sys
from collections import deque
sys.setrecursionlimit(2000000)
n, m = map(int, input().split())
scores = list(map(int, input().split()))
adj = [[] for _ in range(n)]
rev_adj = [[] for _ in range(n)]
for _ in range(m):
    a, b = map(int, input().split())
    a -= 1
    b -= 1
    adj[a].append(b)
    rev_adj[b].append(a)
# ----- Kosaraju -----
visited = [False] * n
stack = []
def dfs1(u):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs1(v)
    stack.append(u)
for i in range(n):
    if not visited[i]:
        dfs1(i)
scc_id = [-1] * n
sccs = []
def dfs2(u, idx):
    scc_id[u] = idx
    sccs[idx].append(u)
    for v in rev_adj[u]:
        if scc_id[v] == -1:
            dfs2(v, idx)
while stack:
    u = stack.pop()
    if scc_id[u] == -1:
        sccs.append([])
        dfs2(u, len(sccs) - 1)
# ----- SCC 权值 -----
score = [0] * len(sccs)
maxsccs = [0] * len(sccs)
for i in range(len(sccs)):
    for node in sccs[i]:

```

```

        score[i] += scores[node]
        maxscs[i] = max(maxscs[i], scores[node])
# ----- 缩点 DAG -----
dag = [[] for _ in range(len(scs))]
indeg = [0] * len(scs)
edges = set()
for u in range(n):
    for v in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a != b and (a, b) not in edges:
            edges.add((a, b))
            dag[a].append(b)
            indeg[b] += 1
# ----- DAG DP -----
dp = score[:]          # 最大亮度和
maxnum = maxscs[:]     # 对应路径上的最大单点亮度
q = deque()
for i in range(len(scs)):
    if indeg[i] == 0:
        q.append(i)
while q:
    u = q.popleft()
    for v in dag[u]:
        cand_sum = dp[u] + score[v]
        cand_max = max(maxnum[u], maxscs[v])
        if cand_sum > dp[v]:
            dp[v] = cand_sum
            maxnum[v] = cand_max
        elif cand_sum == dp[v]:
            if cand_max > maxnum[v]:
                maxnum[v] = cand_max
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
ans_sum = 0
ans_max = 0
for i in range(len(scs)):
    if dp[i] > ans_sum:
        ans_sum = dp[i]
        ans_max = maxnum[i]
    elif dp[i] == ans_sum:

```



```
ans_max = max(ans_max, maxnum[i])
print(ans_sum, ans_max)
```

P4017 最大食物链计数

题目背景

你知道食物链吗？Delia 生物考试的时候，数食物链条数的题目全都错了，因为她总是重复数了几条或漏掉了几条。于是她来就来求助你，然而你也不会啊！写一个程序来帮帮她吧。

题目描述

给你一个食物网，你要求出这个食物网中最大食物链的数量。

（这里的“最大食物链”，指的是生物学意义上的食物链，即最左端是不会捕食其他生物的生产者，最右端是不会被其他生物捕食的消费者。）

Delia 非常急，所以你只有 1 秒的时间。

由于这个结果可能过大，你只需要输出总数模上 80112002 的结果。

输入格式

第一行，两个正整数 n 、 m ，表示生物种类 n 和吃与被吃的关系数 m 。

接下来 m 行，每行两个正整数，表示被吃的生物 A 和吃 A 的生物 B。

输出格式

一行一个整数，为最大食物链数量模上 80112002 的结果。

输入输出样例 #1

输入 #1

```
5 7
1 2
1 3
2 3
3 5
2 5
4 5
3 4
```

输出 #1

```
5
```

说明/提示

数据中不会出现环，满足生物学的要求。（感谢 @AKEE）

```

from collections import deque
n,m = map(int,input().split())
graph = [[] for _ in range(n)]
for _ in range(m):
    i,j = map(int,input().split())
    i -= 1
    j -= 1
    graph[i].append(j)
dp = [0 for _ in range(n)]
def karn(graph,n): # n是顶点数
    indegree = [0] * n
    result = []
    queue = deque()
    # 计算每个顶点的入度
    myset = set()
    for u in range(n):
        for v in graph[u]:
            indegree[v] += 1
    # 将入度为 0 的顶点加入队列
    for u in range(n):
        if indegree[u] == 0:
            myset.add(u)
            queue.append(u)
    # 执行拓扑排序
    while queue:
        u = queue.popleft()
        result.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    result.reverse()
    for num in result:
        for t in graph[num]:
            dp[num] += dp[t]
            dp[num] %= 80112002
        dp[num] = max(dp[num],1)
    total = 0
    for i in myset:
        total += dp[i]
        total %= 80112002

```

```
print(total)
karn(graph,n)
```

P1262 [POI 1996 R3] 间谍网络

题目描述

由于外国间谍的大量渗入，国家安全正处于高度的危机之中。如果 A 间谍手中掌握着关于 B 间谍的犯罪证据，则称 A 可以揭发 B。有些间谍收受贿赂，只要给他们一定数量的美元，他们就愿意交出手中掌握的全部情报。所以，如果我们能够收买一些间谍的话，我们就可能控制间谍网中的每一分子。因为一旦我们逮捕了一个间谍，他手中掌握的情报都将归我们所有，这样就有可能逮捕新的间谍，掌握新的情报。

我们的反间谍机关提供了一份资料，包括所有已知的受贿的间谍，以及他们愿意收受的具体数额。同时我们还知道哪些间谍手中具体掌握了哪些间谍的资料。假设总共有 n 个间谍 (n 不超过 3000)，每个间谍分别用 1 到 3000 的整数来标识。

请根据这份资料，判断我们是否有可能控制全部的间谍，如果可以，求出我们所需要支付的最少资金。否则，输出不能被控制的一个间谍。

输入格式

第一行只有一个整数 n 。

第二行是整数 p 。表示愿意被收买的人数， $1 \leq p \leq n$ 。

接下来的 p 行，每行有两个整数，第一个数是一个愿意被收买的间谍的编号，第二个数表示他将会被收买的数额。这个数额不超过 20000。

紧跟着一行只有一个整数 r ， $1 \leq r \leq 8000$ 。然后 r 行，每行两个正整数，表示数对 (A, B) ，A 间谍掌握 B 间谍的证据。

输出格式

如果可以控制所有间谍，第一行输出 YES，并在第二行输出所需要支付的贿金最小值。否则输出 NO，并在第二行输出不能控制的间谍中，编号最小的间谍编号。

输入输出样例 #1

输入 #1

```
3
2
1 10
2 100
2
1 3
2 3
```

输出 #1

YES

110

输入输出样例 #2

输入 #2

4

2

1 100

4 200

2

1 2

3 4

输出 #2

NO

3

```

import sys
from collections import deque
sys.setrecursionlimit(10000)
n = int(input())
p = int(input())
costs = {}
for _ in range(p):
    a,b = map(int,input().split())
    costs[a-1] = b
q = int(input())
adj = [[] for _ in range(n)]
for _ in range(q):
    a,b = map(int,input().split())
    a -= 1
    b -= 1
    adj[a].append(b)
def kosaraju(n, adj):
    # 1. 正向 DFS, 记录完成顺序
    visited = [False] * n
    stack = []
    def dfs1(u):
        visited[u] = True
        for v in adj[u]:
            if not visited[v]:
                dfs1(v)
        stack.append(u) # 回溯时压入栈
    for i in range(n):
        if not visited[i]:
            dfs1(i)
    # 2. 创建反向图
    rev_adj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            rev_adj[v].append(u)
    # 3. 反向 DFS, 提取 SCC
    visited = [False] * n
    sccs = []
    def dfs2(u, current_scc):
        visited[u] = True
        current_scc.append(u)
        for v in rev_adj[u]:
            if not visited[v]:
                dfs2(v, current_scc)

```

```

while stack:
    u = stack.pop()
    if not visited[u]:
        current_scc = []
        dfs2(u, current_scc)
        sccs.append(current_scc)
    return sccs
sccs = kosaraju(n,adj)
# 给每个点分配SCC编号
scc_id = [0] * n
for i in range(len(sccs)):
    for node in sccs[i]:
        scc_id[node] = i
dag = [[] for _ in range(len(sccs))]
in_degree = [0] * len(sccs)
edges = set() # 防止重复建边
for u in range(n):
    for v in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a != b and (a, b) not in edges:
            edges.add((a, b))
            dag[a].append(b)
            in_degree[b] += 1
zero_in = []
for i in range(len(sccs)):
    if in_degree[i] == 0:
        zero_in.append(i)
minsum = 0
for num in zero_in:
    total = float("inf")
    for t in sccs[num]:
        if t in costs:
            total = min(total, costs[t])
    if total != float("inf"):
        minsum += total
avai = set()
avail = set()
for i in costs:
    avai.add(scc_id[i])
for i in avai:
    q = deque()
    q.append(i)

```

```

while q:
    t = q.popleft()
    avail.add(t)
    for s in dag[t]:
        if s not in avail:
            q.append(s)
unavai = []
for i in range(len(sccs)):
    if i not in avail:
        unavai.extend(sccs[i])
if len(unavai) != 0:
    print("NO")
    print(min(unavai)+1)
else:
    print("YES")
    print(minsum)

```

P1991 无线通讯网

题目描述

国防部计划用无线网络连接若干个边防哨所。2 种不同的通讯技术用来搭建无线网络；

每个边防哨所都要配备无线电收发器；有一些哨所还可以增配卫星电话。

任意两个配备了一条卫星电话线路的哨所（两边都有卫星电话）均可以通话，无论他们相距多远。而只通过无线电收发器通话的哨所之间的距离不能超过 D ，这是受收发器的功率限制。收发器的功率越高，通话距离 D 会更远，但同时价格也会更贵。

收发器需要统一购买和安装，所以全部哨所只能选择安装一种型号的收发器。换句话说，每一对哨所之间的通话距离都是同一个 D 。你的任务是确定收发器必须的最小通话距离 D ，使得每一对哨所之间至少有一条通话路径（直接的或者间接的）。

输入格式

第一行，2 个整数 S 和 P ， S 表示可安装的卫星电话的哨所数， P 表示边防哨所的数量。

接下里 P 行，每行两个整数 x, y 描述一个哨所的平面坐标 (x, y) ，以 km 为单位。

输出格式

第一行，1 个实数 D ，表示无线电收发器的最小传输距离，精确到小数点后两位。

输入输出样例 #1

输入 #1

2 4
0 100
0 300
0 600
150 750

输出 #1

212.13

说明/提示

数据范围及约定

- 对于 20% 的数据: $P = 2, S = 1$;
- 对于另外 20% 的数据: $P = 4, S = 2$;
- 对于 100% 的数据保证: $1 \leq S \leq 100, S < P \leq 500, 0 \leq x, y \leq 10000$ 。


```

s,p = map(int,input().split())
mylist = []
for _ in range(p):
    x,y = map(int,input().split())
    mylist.append((x,y))
edges = []
def dist(i,j):
    a,b = mylist[i]
    c,d = mylist[j]
    return (a-c)**2 + (b-d)**2
graph = [[0]*p for _ in range(p)]
for i in range(p):
    for j in range(i,p):
        distance = dist(i,j)
        edges.append((distance,i,j))
        edges.append((distance,j,i))
        graph[i][j] = distance
        graph[j][i] = distance
def kruskal(n, edges):
    # 按权重从小到大排序
    edges.sort()
    # 初始化 parent 数组 (代替类的 self.parent)
    parent = list(range(n))
    parent = [i for i in range(n)]
    cc = n
    def find(i):
        if parent[i] != i:
            parent[i] = find(parent[i]) # 路径压缩
        return parent[i]
    def union(i, j):
        nonlocal cc
        root_i = find(i)
        root_j = find(j)
        if root_i != root_j:
            parent[root_i] = root_j
            cc -= 1
    for w,u,v in edges:
        union(u, v)
        if cc <= s:
            res = w
            break

```

```
print(f"{res**(.5):.2f}")
kruskal(p, edges)
```

P5960 【模板】差分约束

题目描述

给出一组包含 m 个不等式，有 n 个未知数的形如：

$$\begin{cases} x_{c_1} - x_{c'_1} \leq y_1 \\ x_{c_2} - x_{c'_2} \leq y_2 \\ \dots \\ x_{c_m} - x_{c'_m} \leq y_m \end{cases}$$

的不等式组，求任意一组满足这个不等式组的解。

输入格式

第一行为两个正整数 n, m ，代表未知数的数量和不等式的数量。

接下来 m 行，每行包含三个整数 c, c', y ，代表一个不等式 $x_c - x_{c'} \leq y$ 。

输出格式

一行， n 个数，表示 $x_1, x_2 \cdots x_n$ 的一组可行解，如果有多组解，请输出任意一组，无解请输出 NO。

输入输出样例 #1

输入 #1

```
3 3
1 2 3
2 3 -2
1 3 1
```

输出 #1

```
5 3 5
```

```

from collections import deque
import sys
input = sys.stdin.readline
n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]
for _ in range(m):
    c, cp, y = map(int, input().split())
    # x_c - x_cp <= y
    # x_c <= x_cp + y
    # cp -> c, weight = y
    g[cp].append((c, y))
dist = [0] * (n + 1)
cnt = [1] * (n + 1)
inq = [True] * (n + 1)
q = deque(range(1, n + 1))
while q:
    u = q.popleft()
    inq[u] = False
    du = dist[u]
    for v, w in g[u]:
        if dist[v] > du + w:
            dist[v] = du + w
            if not inq[v]:
                q.append(v)
                inq[v] = True
                cnt[v] += 1
                if cnt[v] > n:
                    print("NO")
                    sys.exit()
print(*dist[1:])

```

P1983 [NOIP 2013 普及组] 车站分级

题目描述

一条单向的铁路线上，依次有编号为 $1, 2, \dots, n$ 的 n 个火车站。每个火车站都有一个级别，最低为 1 级。现有若干趟车次在这条线路上行驶，每一趟都满足如下要求：如果这趟车次停靠了火车站 x ，则始发站、终点站之间所有级别大于等于火车站 x 的都必须停靠。

注意：起始站和终点站自然也算作事先已知需要停靠的站点。

例如，下表是 5 趟车次的运行情况。其中，前 4 趟车次均满足要求，而第 5 趟车次由于停靠了 3 号火车站（2 级）却未停靠途经的 6 号火车站（亦为 2 级）而不满足要求。

车站编号	1		2		3		4		5		6		7		8		9
车站级别	3		1		2		1		3		2		1		1		3
车次																	
1	始	→	→	→	停	→	→	→	停	→	终						
2					始	→	→	→	停	→	终						
3	始	→	→	→	→	→	→	→	停	→	→	→	→	→	→	→	终
4							始	→	停	→	停	→	停	→	停	→	终
5					始	→	→	→	停	→	→	→	→	→	→	→	终

现有 m 趟车次的运行情况（全部满足要求），试推算这 n 个火车站至少分为几个不同的级别。

输入格式

第一行包含 2 个正整数 n, m ，用一个空格隔开。

第 $i + 1$ 行 ($1 \leq i \leq m$) 中，首先是一个正整数 s_i ($2 \leq s_i \leq n$)，表示第 i 趟车次有 s_i 个停靠站；接下来有 s_i 个正整数，表示所有停靠站的编号，从小到大排列。每两个数之间用一个空格隔开。输入保证所有的车次都满足要求。

输出格式

一个正整数，即 n 个火车站最少划分的级别数。

输入输出样例 #1

输入 #1

```

9 2
4 1 3 5 6
3 3 5 6

```

输出 #1

```

2

```

输入输出样例 #2

输入 #2

```

9 3
4 1 3 5 6
3 3 5 6
3 1 5 9

```

输出 #2

```

3

```

说明/提示

对于 20% 的数据, $1 \leq n, m \leq 10$;

对于 50% 的数据, $1 \leq n, m \leq 100$;

对于 100% 的数据, $1 \leq n, m \leq 1000$ 。

```

import sys
from collections import deque
data = iter(sys.stdin.read().split())
n,m = int(next(data)),int(next(data))
graph = [[] for _ in range(n+m)]
indegree = [0] * (n+m)
for i in range(m):
    t = int(next(data))
    mylist = [int(next(data))-1 for _ in range(t)]
    myset = set(mylist)
    for j in range(mylist[0],mylist[-1]+1):
        if j not in myset:
            graph[j].append(n+i)
            indegree[n+i] += 1
        else:
            graph[n+i].append(j)
            indegree[j] += 1
def karn(graph): # n是顶点数
    result = []
    queue = deque()
    # 将入度为 0 的顶点加入队列
    for u in range(n+m):
        if indegree[u] == 0:
            if u < n:
                queue.append(u)
            else:
                for r in graph[u]:
                    indegree[r] -= 1
                    if indegree[r] == 0:
                        queue.append(r)
    count = 0
    # 执行拓扑排序
    while queue:
        count += 1
        for _ in range(len(queue)):
            u = queue.popleft()
            result.append(u)
            for v in graph[u]:
                indegree[v] -= 1
                if indegree[v] == 0:
                    if v >= n:
                        for r in graph[v]:
                            indegree[r] -= 1

```

```

        if indegree[r] == 0:
            queue.append(r)
        else:
            queue.append(v)

    return count
print(karn(graph))

```

07735:道路

总Time Limit: 1000ms Memory Limit: 65536kB

Description

N个以 1 ... N 标号的城市通过单向的道路相连:。每条道路包含两个参数: 道路的长度和需要为该路付的通行费 (以 Bob and Alice 过去住在城市 1.在注意到Alice在他们过去喜欢玩的纸牌游戏中作弊后, Bob和她分手了, 并且决

Input

第一行包含一个整数K, $0 \leq K \leq 10000$, 代表Bob能够在他路上花费的最大的金币数。第二行包含整数N, $2 \leq$ 接下来的R行, 每行具体指定几个整数S, D, L 和 T来说明关于道路的一些情况, 这些整数之间通过空格间隔:

S is 道路起始城市, $1 \leq S \leq N$

D is 道路终点城市, $1 \leq D \leq N$

L is 道路长度, $1 \leq L \leq 100$

T is 通行费 (以金币数量形式度量), $0 \leq T \leq 100$

注意不同的道路可能有相同的起点和终点。

Output

输入结果应该只包括一行, 即从城市1到城市N所需要的最小的路径长度 (花费不能超过K个金币)。如果这样的路径不存

Sample Input

```

5
6
7
1 2 2 3
2 4 3 3
3 4 2 4
1 3 4 1
4 6 2 1
3 5 2 0
5 4 3 2

```

Sample Output

```

11

```

```

import sys
import heapq
data = iter(sys.stdin.read().split())
k = int(next(data))
n = int(next(data))
graph = [[] for _ in range(n)]
for _ in range(int(next(data))):
    a,b,c,d = int(next(data)),int(next(data)),int(next(data)),int(next(data))
    a -= 1
    b -= 1
    graph[a].append((b,c,d))
def dijkstra(graph, start):
    dist = [[float("inf")]*(k+1) for _ in range(n)]
    dist[start][0] = 0
    pq = [(0,0, start)]
    while pq:
        current_dist,cost1, node = heapq.heappop(pq)
        for neighbor, weight,cost2 in graph[node]:
            newcost = cost1+cost2
            if newcost > k:
                continue
            new_dist = current_dist + weight
            if dist[neighbor][newcost] <= new_dist:
                continue
            dist[neighbor][newcost] = new_dist
            heapq.heappush(pq, (new_dist, newcost,neighbor))
    return min(dist[-1]) if min(dist[-1]) != float("inf") else -1
print(dijkstra(graph,0))

```


02979:陪审团的人选

Description

在遥远的国家佛罗布尼亚，嫌犯是否有罪，须由陪审团决定。陪审团是由法官从公众中挑选的。先随机挑选n个人作为陪审团。

Input
输入包含多组数据。每组数据的第一行是两个整数n和m，n是候选人数目，m是陪审团人数。注意， $1 \leq n \leq 200$ ， $1 \leq m$

Output

对每组数据，先输出一行，表示答案所属的组号，如 'Jury #1', 'Jury #2'，等。接下来的一行要象例子那样输出

Sample Input

4 2

1 2

2 3

4 1

6 2

0 0

Sample Output

Jury #1

Best jury has value 6 for prosecution and value 4 for defence:

2 3

```

import sys
def solve_case(candidates, m):
    n = len(candidates)
    max_diff = 20 * m
    offset = max_diff
    width = max_diff * 2 + 1
    neg_inf = -10**9
    dp = [[neg_inf] * width for _ in range(m + 1)]
    # dp[j][idx] 选 j 个人, 总 diff = idx - offset 时最大的总 sum
    dp[0][offset] = 0 # 选0个人, diff=0, sum=0
    # take[i][j][d] == 1 means candidate i is used when reaching state (j, d)
    # after considering candidates 1..i. bytearray keeps this compact enough.
    take = [[bytearray(width) for _ in range(m + 1)] for _ in range(n + 1)]
    for i, (prosecution, defence) in enumerate(candidates, start=1):
        diff = prosecution - defence
        total = prosecution + defence
        next_dp = [row[:] for row in dp]
        upper = min(i, m)
        for chosen in range(1, upper + 1):
            prev_row = dp[chosen - 1]
            curr_row = next_dp[chosen]
            for old_idx, old_total in enumerate(prev_row):
                if old_total == neg_inf:
                    continue
                new_idx = old_idx + diff
                if 0 <= new_idx < width and old_total + total > curr_row[new_idx]:
                    curr_row[new_idx] = old_total + total
                    take[i][chosen][new_idx] = 1
            dp = next_dp
    best_idx = -1
    for delta in range(max_diff + 1):
        left = offset - delta
        right = offset + delta
        left_total = dp[m][left] if 0 <= left < width else neg_inf
        right_total = dp[m][right] if 0 <= right < width else neg_inf
        if left_total == neg_inf and right_total == neg_inf:
            continue
        if right_total > left_total:
            best_idx = right
        else:
            best_idx = left
        break
    selected = []

```

```

chosen = m
idx = best_idx
for i in range(n, 0, -1):
    if chosen > 0 and take[i][chosen][idx]:
        selected.append(i)
        prosecution, defence = candidates[i - 1]
        idx -= prosecution - defence
        chosen -= 1
selected.reverse()
prosecution_sum = sum(candidates[i - 1][0] for i in selected)
defence_sum = sum(candidates[i - 1][1] for i in selected)
return prosecution_sum, defence_sum, selected

def main():
    data = sys.stdin.read().split()
    pos = 0
    case_no = 1
    output = []
    while pos < len(data):
        n = int(data[pos])
        m = int(data[pos + 1])
        pos += 2
        if n == 0 and m == 0:
            break
        candidates = []
        for _ in range(n):
            prosecution = int(data[pos])
            defence = int(data[pos + 1])
            pos += 2
            candidates.append((prosecution, defence))
        prosecution_sum, defence_sum, selected = solve_case(candidates, m)
        output.append(f"Jury #{case_no}")
        output.append(
            f"Best jury has value {prosecution_sum} for prosecution "
            f"and value {defence_sum} for defence:"
        )
        output.append(" " + " ".join(map(str, selected)))
        output.append("")
        case_no += 1
    sys.stdout.write("\n".join(output))

if __name__ == "__main__":
    main()

```

P2515 软件安装

现在我们的手头有 N 个软件，对于一个软件 i ，它要占用 W_i 的磁盘空间，它的价值为 V_i 。我们希望能从中选择一些软件安装到一台磁盘容量为 M 计算机上，使得这些软件的价值尽可能大（即 V_i 的和最大）。

但是现在有个问题：软件之间存在依赖关系，即软件 i 只有在安装了软件 j （包括软件 j 的直接或间接依赖）的情况下才能正确工作（软件 i 依赖软件 j ）。幸运的是，一个软件最多依赖另外一个软件。如果一个软件不能正常工作，那么它能够发挥的作用为 0。

我们现在知道了软件之间的依赖关系：软件 i 依赖软件 D_i 。现在请你设计出一种方案，安装价值尽量大的软件。一个软件只能被安装一次，如果一个软件没有依赖则 $D_i = 0$ ，这时只要这个软件安装了，它就能正常工作。

输入格式

第 1 行： $N, M (0 \leq N \leq 100, 0 \leq M \leq 500)$

第 2 行： $W_1, W_2, \dots, W_i, \dots, W_n (0 \leq W_i \leq M)$

第 3 行： $V_1, V_2, \dots, V_i, \dots, V_n (0 \leq V_i \leq 1000)$

第 4 行： $D_1, D_2, \dots, D_i, \dots, D_n (0 \leq D_i \leq N, D_i \neq i)$

输出格式

一个整数，代表最大价值。

输入输出样例 #1

输入 #1

```
3 10
5 5 6
2 3 4
0 1 1
```

输出 #1

```
5
```

```

import sys
def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = iter(data)
    n = int(next(it))
    m = int(next(it))
    weights = [0] + [int(next(it)) for _ in range(n)]
    values = [0] + [int(next(it)) for _ in range(n)]
    deps = [0] + [int(next(it)) for _ in range(n)]
    graph = [[] for _ in range(n + 1)]
    for i in range(1, n + 1):
        if deps[i] != 0:
            graph[deps[i]].append(i)
    dfn = [0] * (n + 1)
    low = [0] * (n + 1)
    in_stack = [False] * (n + 1)
    stack = []
    comp_id = [0] * (n + 1)
    comp_count = 0
    timer = 0
    sys.setrecursionlimit(1000000)
    def tarjan(u):
        nonlocal timer, comp_count
        timer += 1
        dfn[u] = low[u] = timer
        stack.append(u)
        in_stack[u] = True
        for v in graph[u]:
            if dfn[v] == 0:
                tarjan(v)
                low[u] = min(low[u], low[v])
            elif in_stack[v]:
                low[u] = min(low[u], dfn[v])
        if low[u] == dfn[u]:
            comp_count += 1
            while True:
                x = stack.pop()
                in_stack[x] = False
                comp_id[x] = comp_count
                if x == u:
                    break

```

```

for i in range(1, n + 1):
    if dfn[i] == 0:
        tarjan(i)
comp_w = [0] * (comp_count + 1)
comp_v = [0] * (comp_count + 1)
indeg = [0] * (comp_count + 1)
tree = [[] for _ in range(comp_count + 1)]
for i in range(1, n + 1):
    cid = comp_id[i]
    comp_w[cid] += weights[i]
    comp_v[cid] += values[i]
seen_edges = set()
for i in range(1, n + 1):
    if deps[i] == 0:
        continue
    parent = comp_id[deps[i]]
    child = comp_id[i]
    if parent != child and (parent, child) not in seen_edges:
        seen_edges.add((parent, child))
        tree[parent].append(child)
        indeg[child] += 1
root = 0
tree.append([])
comp_w.append(0)
comp_v.append(0)
for cid in range(1, comp_count + 1):
    if indeg[cid] == 0:
        tree[root].append(cid)
neg_inf = -10**18
def dfs(u):
    # dp[c]表示已经选了节点 u (对应 SCC), 总容量为 c 时, 从 u 的整棵子树能获得的最大价值。
    dp = [neg_inf] * (m + 1)
    for cap in range(comp_w[u], m + 1):
        dp[cap] = comp_v[u]
    for v in tree[u]:
        child = dfs(v)
        merged = dp[:]
        for cap in range(m + 1):
            if dp[cap] == neg_inf:
                continue
            rest = m - cap
            for extra in range(rest + 1):
                if child[extra] != neg_inf:

```

```
        total_cap = cap + extra
        merged[total_cap] = max(
            merged[total_cap],
            dp[cap] + child[extra],
        )
    dp = merged
    return dp
print(dfs(root)[m])
if __name__ == "__main__":
    main()
```

30921:猫猫搭积木

猫猫爱搭积木。

猫猫一共有 n 块积木。每次猫猫会选择两块积木 x,y ，并且将它们所在的积木堆搭起来。不幸的是，一堆积木中如果和

Input

第一行包含三个整数 n,q,s ($1 \leq n \leq 10^5, 0 \leq q \leq 5 * 10^5, 2 \leq s \leq 10^9$)，表示积木数目、操作次数和

接下来 q 行，每行包含两个整数 x_i,y_i ($1 \leq x_i,y_i \leq n$)，表示一次操作。

Output

输出 q 行，每行一个整数，表示第 i 次操作后的总积木堆数。

Sample Input

```
5 10 3
1 2
4 2
1 5
2 3
4 1
5 1
2 4
1 3
2 5
3 4
```

Sample Output

```
4
5
4
3
4
3
4
5
4
3
```



```

import sys
def main():
    data = sys.stdin.read().split()
    it = iter(data)
    n = int(next(it))
    q = int(next(it))
    s = int(next(it))
    parent = list(range(n + 1))
    size = [1] * (n + 1)
    members = [[] for _ in range(n + 1)]
    for i in range(1, n + 1):
        members[i].append(i)

    pile_count = n
    out = []

    def find(i):
        if (parent[i] == i):
            return i
        else:
            result = find(parent[i])
            parent[i] = result
            return result

    for _ in range(q):
        x = int(next(it))
        y = int(next(it))
        rx = find(x)
        ry = find(y)

        if rx != ry:
            if len(members[rx]) < len(members[ry]):
                rx, ry = ry, rx
            parent[ry] = rx
            members[rx].extend(members[ry])
            members[ry].clear()
            size[rx] += size[ry]
            pile_count -= 1
            if size[rx] >= s:
                collapsed = members[rx]
                pile_count += size[rx] - 1
                for block in collapsed:
                    parent[block] = block

```

```
        size[block] = 1
        members[block] = [block]
    collapsed.clear()
```

```
    out.append(str(pile_count))
```

```
sys.stdout.write("\n".join(out))
```

```
if __name__ == "__main__":
    main()
```

30868:upstairs

总Time Limit: 1000ms Memory Limit: 65536kB

Description

有一层楼一共有 n 层，小 z 从第 0 层出发，想要前往某一层。上楼的方式只有一个奇怪的电梯，这个电梯每次只能形式化地，对于给定的 a, b, c ，是否存在自然数解 (x, y, z) ，使得对于某个询问 h_i ，满足 $h_i = ax + by + cz$ 。

数据范围：

对于全部数据， $0 \leq h_i \leq n$ ， $0 \leq q \leq 10^5$ ， $1 \leq n \leq 10^{18}$ ， $0 \leq a, b, c \leq 10^6$ 。

对于 1% 的数据，为样例。

对于另外 9% 的数据， $a=0$ ， $0 \leq b, c \leq 10^3$ ， $1 \leq q \leq 10^3$ ， $1 \leq n \leq 10^5$ 。

对于 30% 的数据， $0 \leq a, b, c \leq 10^3$ ， $1 \leq q \leq 10^3$ ， $1 \leq n \leq 10^9$ 。

对于 50% 的数据， $0 \leq a, b, c \leq 3 * 10^3$ ，且 b, c 互质。

对于 70% 的数据， $0 \leq a \leq 10^4$ 。

Input

一共 $q+2$ 行。

第一行有 3 个数，分别代表 a, b, c 。

第二行有 1 个数 q ，表示接下来有 q 组询问。

对第 3 行至第 $q+2$ 行，每行一个数 $h_i (1 \leq i \leq q)$ ，表示小 z 想要前往的楼层数。

Output

一共 q 行，每行一个字符串。

如果小 z 能够到达，输出 Yes，否则输出 No。

Sample Input

sample1 input:

7 3 5

4

4

10

13

41

sample1 output:

No

Yes

Yes

Yes

解释：可以证明，第 4 层无法到达，而 $10 = 5+5$ ， $13 = 7+3+3$ ， $41 = 7+7+7+7+7+3+3$ 。

到达该层的方案不唯一。

Sample Output

sample2 input:

13 53 39

5

414

690

597

390

144

sample2 output:

No

Yes

No

Yes

Yes

```

import sys
import heapq
def main():
    data = sys.stdin.read().strip().split()
    it = iter(data)
    a = int(next(it))
    b = int(next(it))
    c = int(next(it))
    q = int(next(it))
    if q == 0:
        return
    queries = [int(next(it)) for _ in range(q)]
    coins = []
    for v in (a, b, c):
        if v > 0:
            coins.append(v)
    coins = sorted(set(coins))
    if not coins:
        out = []
        for h in queries:
            out.append("Yes" if h == 0 else "No")
        print("\n".join(out_lines))
        return
    def gcd(x, y):
        while y:
            x, y = y, x % y
        return x
    g = coins[0]
    for v in coins[1:]:
        g = gcd(g, v)
    if len(coins) == 1:
        step = coins[0]
        out = []
        for h in queries:
            if h == 0 or (step > 0 and h % step == 0):
                out.append("Yes")
            else:
                out.append("No")
        print("\n".join(out_lines))
        return
    # Scale by gcd so the gcd becomes 1 for the residue calculations.
    scaled_coins = [v // g for v in coins]
    smallest = scaled_coins[0]

```

```

# Dijkstra on residues modulo smallest coin.
INF = 10**30
dist = [INF] * smallest
dist[0] = 0
heap = [(0, 0)]
edges = [v for v in scaled_coins if v != smallest]
while heap:
    d, r = heapq.heappop(heap)
    if d != dist[r]:
        continue
    for w in edges:
        nr = (r + w) % smallest
        nd = d + w
        if nd < dist[nr]:
            dist[nr] = nd
            heapq.heappush(heap, (nd, nr))
out_lines = []
for h in queries:
    if h == 0:
        out_lines.append("Yes")
        continue
    if h % g != 0:
        out_lines.append("No")
        continue
    h_scaled = h // g
    r = h_scaled % smallest
    if h_scaled >= dist[r]:
        out_lines.append("Yes")
    else:
        out_lines.append("No")
print("\n".join(out_lines))
if __name__ == '__main__':
    main()

```

02186 Popular Cows

总Time Limit: 2000ms Memory Limit: 65536kB

Description

Every cow's dream is to become the most popular cow in the herd. In a herd of N ($1 \leq N$) popular, even if this is not explicitly specified by an ordered pair in the input. Your

Input

* Line 1: Two space-separated integers, N and M

* Lines 2.. M : Two space-separated numbers A and B , meaning that A thinks B is popular

Output

* Line 1: A single integer that is the number of cows who are considered popular by eve

Sample Input

3 3

1 2

2 1

2 3

Sample Output

1

注意这里出度为0的必须只有一个才行，否则就没有popular的cow

```

n,m = map(int,input().split())
import sys
sys.setrecursionlimit(100000)
from collections import defaultdict
adj = defaultdict(list)
for _ in range(m):
    i,j = map(int,input().split())
    i -= 1
    j -= 1
    adj[i].append(j)
visited = [False] * n
stack = []
def dfs1(u):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs1(v)
    stack.append(u)
for i in range(n):
    if not visited[i]:
        dfs1(i)
# 2. 创建反向图
rev_adj = [[] for _ in range(n)]
for u in range(n):
    for v in adj[u]:
        rev_adj[v].append(u)
# 3. 反向 DFS, 提取 SCC
visited = [False] * n
index = 0
scc = [0 for _ in range(n)]
count = [0 for _ in range(n)]
def dfs2(u,index):
    scc[u] = index
    count[index] += 1
    visited[u] = True
    for v in rev_adj[u]:
        if not visited[v]:
            dfs2(v,index)
while stack:
    u = stack.pop()
    if not visited[u]:
        dfs2(u,index)
        index += 1

```



```
chudu = [0 for _ in range(index)]
for i in range(n):
    for j in adj[i]:
        if scc[i] == scc[j]:
            continue
        else:
            chudu[scc[i]] += 1
total = 0
visit = 0
for i in range(index):
    if chudu[i] == 0:
        visit += 1
        total = count[i]
if visit == 1:
    print(total)
else:
    print(0)
```

3629 Minimum Jumps to Reach End via Prime Teleportation

You are given an integer array `nums` of length `n`.

You start at index `0`, and your goal is to reach index `n - 1`.

From any index `i`, you may perform one of the following operations:

Adjacent Step: Jump to index `i + 1` or `i - 1`, if the index is within bounds.

Prime Teleportation: If `nums[i]` is a prime number `p`, you may instantly jump to any index `j` such that `nums[j] = p`.

Return the minimum number of jumps required to reach index `n - 1`.

Example 1:

Input: `nums = [1,2,4,6]`

Output: 2

Explanation:

One optimal sequence of jumps is:

Start at index `i = 0`. Take an adjacent step to index `1`.

At index `i = 1`, `nums[1] = 2` is a prime number. Therefore, we teleport to index `i = 3` as `nums[3] = 6`.

Thus, the answer is 2.

Example 2:

Input: `nums = [2,3,4,7,9]`

Output: 2

Explanation:

One optimal sequence of jumps is:

Start at index `i = 0`. Take an adjacent step to index `i = 1`.

At index `i = 1`, `nums[1] = 3` is a prime number. Therefore, we teleport to index `i = 4` as `nums[4] = 9`.

Thus, the answer is 2.

Example 3:

Input: `nums = [4,6,5,8]`

Output: 3

Explanation:

Since no teleportation is possible, we move through `0 → 1 → 2 → 3`. Thus, the answer is 3.

```

from collections import defaultdict, deque
class Solution:
    def minJumps(self, nums) -> int:
        n = len(nums)
        if n == 1:
            return 0
        MAXV = max(nums)
        # 欧拉筛 + 最小质因子
        spf = [0] * (MAXV + 1)
        primes = []
        for i in range(2, MAXV + 1):
            if spf[i] == 0:
                spf[i] = i
                primes.append(i)
            for p in primes:
                if p > spf[i] or i * p > MAXV:
                    break
                spf[i * p] = p
        # 判断质数
        def is_prime(x):
            return x >= 2 and spf[x] == x
        # 质因数分解
        def get_factors(x):
            factors = set()
            while x > 1:
                p = spf[x]
                factors.add(p)
                while x % p == 0:
                    x //= p
            return factors
        prime_to_indices = defaultdict(list)
        for i, x in enumerate(nums):
            for p in get_factors(x):
                prime_to_indices[p].append(i)
        # BFS
        q = deque([0])
        dist = [-1] * n
        dist[0] = 0
        used_prime = set()
        while q:
            i = q.popleft()
            step = dist[i] + 1
            if i == n - 1:

```

```

        return dist[i]
# 相邻跳
for ni in (i - 1, i + 1):
    if 0 <= ni < n and dist[ni] == -1:
        dist[ni] = step
        q.append(ni)
# teleport
x = nums[i]
if is_prime(x) and x not in used_prime:
    for ni in prime_to_indices[x]:
        if dist[ni] == -1:
            dist[ni] = step
            q.append(ni)
    used_prime.add(x)
return -1

```

3660 Jump Game IX

You are given an integer array `nums`.

From any index `i`, you can jump to another index `j` under the following rules:

Jump to index `j` where `j > i` is allowed only if `nums[j] < nums[i]`.

Jump to index `j` where `j < i` is allowed only if `nums[j] > nums[i]`.

For each index `i`, find the maximum value in `nums` that can be reached by following any sequence of jumps.

Return an array `ans` where `ans[i]` is the maximum value reachable starting from index `i`.

Example 1:

Input: `nums = [2,1,3]`

Output: `[2,2,3]`

Explanation:

For `i = 0`: No jump increases the value.

For `i = 1`: Jump to `j = 0` as `nums[j] = 2` is greater than `nums[i]`.

For `i = 2`: Since `nums[2] = 3` is the maximum value in `nums`, no jump increases the value.

Thus, `ans = [2, 2, 3]`.

Example 2:

Input: `nums = [2,3,1]`

Output: `[3,3,3]`

Explanation:

For `i = 0`: Jump forward to `j = 2` as `nums[j] = 1` is less than `nums[i] = 2`, then from `i = 2` jump to `j = 1` as `nums[j] = 3` is greater than `nums[i] = 1`.

For `i = 1`: Since `nums[1] = 3` is the maximum value in `nums`, no jump increases the value.

For `i = 2`: Jump to `j = 1` as `nums[j] = 3` is greater than `nums[2] = 1`.

Thus, `ans = [3, 3, 3]`.

```

class Solution:
    def maxValue(self, nums: List[int]) -> List[int]:
        n = len(nums)
        if n == 1:
            return nums
        res = [0 for _ in range(n)]
        from collections import defaultdict
        pre_max = [0 for _ in range(n)]
        suf_min = [0 for _ in range(n)]
        for i in range(n):
            if i >= 1:
                pre_max[i] = max(pre_max[i-1], nums[i])
            else:
                pre_max[i] = nums[0]
        for i in range(n-1, -1, -1):
            if i < n-1:
                suf_min[i] = min(suf_min[i+1], nums[i])
            else:
                suf_min[i] = nums[-1]
        begin, end = 0, 0
        for i in range(n):
            if i == 0:
                if suf_min[i+1] >= nums[i]:
                    res[0] = nums[0]
                continue
            if i == n-1:
                if pre_max[i-1] <= nums[i]:
                    res[i] = nums[i]
                continue
            end = i
            for j in range(begin, end+1):
                res[j] = pre_max[i]
            continue
            if pre_max[i-1] <= suf_min[i+1] and nums[i] < pre_max[i-1]:
                end = i
                for j in range(begin, end+1):
                    res[j] = pre_max[i]
            elif pre_max[i-1] <= suf_min[i+1] and nums[i] > suf_min[i+1]:
                begin, end = i, i
            elif pre_max[i-1] <= suf_min[i+1]:
                res[i] = nums[i]
            else:

```

```
        end = i
    return res
```

27093:排队又来了

有 N 名同学从左到右排成一排，第 i 名同学的身高为 h_i 。现在张老师想改变排队的顺序，他能进行任意多次（包括 0 次）交换相邻同学，并且他们的身高之差不超过 D ，那么老师就能交换他俩的顺序。

请你帮张老师算一算，通过以上操作，字典序最小的所有同学（从左到右）身高序列是什么？

输入

第一行包含两个正整数。接下去 N 行，每行一个正整数 h_i ($1 \leq h_i \leq 10^9$) 表示从左到右每名同学的身高。

输出

输出 N 行，第 i 行表示答案中第 i 名同学的身高。

样例输入

5 3

7

7

3

6

2

样例输出

6

7

7

2

3

```

import sys
import bisect

# 增加递归深度限制（虽然本解法主要通过迭代实现，这在大规模数据下是好习惯）
sys.setrecursionlimit(200000)

def solve():
    # 使用 sys.stdin.read 一次性读取所有输入，显著提升 Python I/O 速度
    input_data = sys.stdin.read().split()

    if not input_data:
        return

    iterator = iter(input_data)

    try:
        N = int(next(iterator))
        D = int(next(iterator))
        # 读取 N 个同学的身高
        h = [int(next(iterator)) for _ in range(N)]
    except StopIteration:
        return

    # --- 1. 离散化 (Coordinate Compression) ---
    # 身高数值范围很大 (1e9)，不能直接作为数组下标。
    # 但 N 只有 1e5，所以将其映射到 [0, M-1] 的排名 (Rank)。
    vals = sorted(list(set(h)))
    rank_map = {v: i for i, v in enumerate(vals)}
    M = len(vals)

    # --- 2. 定义树状数组 (Fenwick Tree) ---
    # bitL: 用于查询值域 [0, val - D - 1] 内的最大层数
    # bitR: 用于查询值域 [val + D + 1, max_val] 内的最大层数
    # 数组大小设为 M + 1（因为树状数组通常使用 1-based 索引）
    bitL = [0] * (M + 1)
    bitR = [0] * (M + 1)

    # 单点更新: 将 idx 位置的值更新为 val（取最大值）
    def bit_update(bit, idx, val):
        limit = len(bit)
        while idx < limit:
            if val > bit[idx]:
                bit[idx] = val

```

```

        idx += idx & (-idx)

# 前缀查询: 查询 [1, idx] 范围内的最大值
def bit_query(bit, idx):
    res = 0
    while idx > 0:
        if bit[idx] > res:
            res = bit[idx]
        idx -= idx & (-idx)
    return res

# --- 3. 计算每个同学的层数 ---
# layer_buckets 用于收集每一层的同学身高
# 字典结构: {层数: [身高1, 身高2, ...]}
layer_buckets = {}
max_layer_global = 0

for val in h:
    r = rank_map[val] # 获取当前身高的离散化排名 (0 ~ M-1)

    # --- 计算左侧阻塞的最大层数 ---

    # 情况 A: 被身高太小的人阻塞 (h_k < val - D)
    # bisect_left 找到第一个 >= val-D 的位置, 其左侧即为 < val-D
    idx_small = bisect.bisect_left(vals, val - D)
    # 查询 bitL 在范围 [0, idx_small-1] 的最大层数 (对应BIT下标 idx_small)
    max_L = bit_query(bitL, idx_small)

    # 情况 B: 被身高太大的人阻塞 (h_k > val + D)
    # bisect_right 找到第一个 > val+D 的位置, 该位置及右侧即为 > val+D
    idx_large = bisect.bisect_right(vals, val + D)

    # 在 bitR 中, 我们使用反转映射技巧:
    # 实际排名 rank 越大, 映射到 bitR 的下标 M - rank 越小。
    # 这样查询 bitR 的前缀最大值, 实际上就是查询原数组的后缀最大值。
    # 范围 [idx_large, M-1] 映射到 bitR 下标 [1, M - idx_large]
    max_R = 0
    if idx_large < M:
        pos_in_bitR = M - idx_large
        max_R = bit_query(bitR, pos_in_bitR)

    # 当前同学的层数 = 所有阻塞者中的最大层数 + 1
    cur_layer = max(max_L, max_R) + 1

```



```

# 记录全局最大层数以便后续输出
if cur_layer > max_layer_global:
    max_layer_global = cur_layer

# 将身高放入对应的层桶中
if cur_layer not in layer_buckets:
    layer_buckets[cur_layer] = []
layer_buckets[cur_layer].append(val)

# --- 更新树状数组 ---
# 将当前同学的信息加入 BIT，成为后续同学可能的阻塞者

# 更新 bitL：正常映射，下标为 r + 1
bit_update(bitL, r + 1, cur_layer)

# 更新 bitR：反转映射，下标为 M - r
bit_update(bitR, M - r, cur_layer)

# --- 4. 输出结果 ---
# 字典序最小原则：
# 1. 必须按层数从小到大输出（拓扑序）
# 2. 同一层内互不阻塞，按身高从小到大输出（贪心）

output = []
for layer in range(1, max_layer_global + 1):
    if layer in layer_buckets:
        # 对该层的所有身高进行排序
        members = sorted(layer_buckets[layer])
        for v in members:
            output.append(str(v))

sys.stdout.write('\n'.join(output) + '\n')

if __name__ == '__main__':
    solve()

```

3331 Find Subtree Sizes After Changes

You are given a tree rooted at node 0 that consists of n nodes numbered from 0 to $n - 1$. You are also given a string s of length n , where $s[i]$ is the character assigned to node i . We make the following changes on the tree one time simultaneously for all nodes x from 1 to $n - 1$: Find the closest node y to node x such that y is an ancestor of x , and $s[x] == s[y]$.

If node y does not exist, do nothing.

Otherwise, remove the edge between x and its current parent and make node y the new parent of x . Return an array `answer` of size n where `answer[i]` is the size of the subtree rooted at node i .

Example 1:

Input: `parent = [-1,0,0,1,1,1]`, `s = "abaabc"`

Output: `[6,3,1,1,1,1]`

Explanation:

The parent of node 3 will change from node 1 to node 0.

Example 2:

Input: `parent = [-1,0,4,0,1]`, `s = "abbba"`

Output: `[5,2,1,1,1]`

Explanation:

The following changes will happen at the same time:

The parent of node 4 will change from node 1 to node 0.

The parent of node 2 will change from node 4 to node 1.

Constraints:

$n == \text{parent.length} == s.\text{length}$

$1 \leq n \leq 10^5$

$0 \leq \text{parent}[i] < n$ for all $i \geq 1$.

`parent[0] == -1`

`parent` represents a valid tree.

`s` consists only of lowercase English letters.

肯定要重新建树，因为一条边的改动会带来一个子树的改变，不单单是一个节点，而追踪每一个节点的最近的value值相同的节点应该就用一个长度为26的list来追踪

```

from typing import List
from collections import defaultdict
class Solution:
    def findSubtreeSizes(self, parent: List[int], s: str) -> List[int]:

        class TreeNode:
            def __init__(self, val, i):
                self.val = val
                self.index = i
                self.children = set()
                self.num_sub = 1
                self.parent = None

        nodes = [TreeNode(s[i], i) for i in range(len(s))]

        letter = [[] for _ in range(26)]

        for i in range(1, len(parent)):
            nodes[parent[i]].children.add(nodes[i])
            nodes[i].parent = nodes[parent[i]]

        stack = []

        def loc(letter):
            return ord(letter)-97

        def dfs2(node):
            """change the tree and return the number of subtree nodes"""
            if not node:
                return 0

            if letter[loc(node.val)]:
                node.parent = letter[loc(node.val)][-1]

            stack.append(node)
            letter[loc(node.val)].append(node)

            for child in node.children:
                dfs2(child)

            stack.pop()
            letter[loc(node.val)].pop()

```

```

dfs2(nodes[0])

parent = [-1]+[nodes[i].parent.index for i in range(1,len(parent))]

nodes = [TreeNode(s[i],i) for i in range(len(s))]
for i in range(1,len(parent)):
    nodes[parent[i]].children.add(nodes[i])

def dfs1(node):
    """return the number of subtree nodes"""
    if not node:
        return 0
    node.num_sub += sum([dfs1(child) for child in node.children])
    return node.num_sub

dfs1(nodes[0])

return [nodes[i].num_sub for i in range(len(s))]

```

Morris 遍历 (Morris Traversal)

$O(1)$ 额外空间、 $O(n)$ 时间

对每个节点 cur:

若 cur 无左子树: 直接访问, 向右走。

若 cur 有左子树: 找到 左子树最右节点 mostRight (中序前驱)。

若 mostRight.right == null: 第一次到 cur → 让 mostRight.right = cur (建线索), cur 向左走。

若 mostRight.right == cur: 第二次到 cur → 恢复 mostRight.right = null (拆线索), cur 向右走。

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
def morris_inorder(root):
    res = []
    cur = root
    while cur:
        # 情况1: 没有左子树, 直接访问, 然后往右走
        if not cur.left:
            res.append(cur.val)
            cur = cur.right
        # 情况2: 有左子树, 找到左子树最右边节点 (前驱)
        else:
            prev = cur.left
            while prev.right and prev.right != cur:
                prev = prev.right
            # 第一次来到 cur: 建立线索, 让 prev.right = cur
            if not prev.right:
                prev.right = cur
                cur = cur.left
            # 第二次来到 cur: 说明左子树遍历完了, 访问 cur, 恢复树
            else:
                prev.right = None
                res.append(cur.val)
                cur = cur.right
    return res
```

```

def morris_preorder(root):
    res = []
    cur = root
    while cur:
        if not cur.left:
            res.append(cur.val)
            cur = cur.right
        else:
            prev = cur.left
            while prev.right and prev.right != cur:
                prev = prev.right
            if not prev.right:
                prev.right = cur
                res.append(cur.val) # 这里提前访问
                cur = cur.left
            else:
                prev.right = None
                cur = cur.right
    return res

```

1123 Lowest Common Ancestor of Deepest Leaves

Given the root of a binary tree, return the lowest common ancestor of its deepest leaf
Recall that:

The node of a binary tree is a leaf if and only if it has no children

The depth of the root of the tree is 0. if the depth of a node is d, the depth of each
The lowest common ancestor of a set S of nodes, is the node A with the largest depth su

Example 1:

Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest leaf-nodes of the tree.

Note that nodes 6, 0, and 8 are also leaf nodes, but the depth of them is 2, but the de
Example 2:

Input: root = [1]

Output: [1]

Explanation: The root is the deepest node in the tree, and it's the lca of itself.

Example 3:

Input: root = [0,1,3,null,2]

Output: [2]

Explanation: The deepest leaf node in the tree is 2, the lca of one node is itself.

学会用递归!!!

```
class Solution :
    def dfs( self, root ) :
        if root == None :
            return None, 0
        leftNode, leftDepth = self.dfs(root.left)
        rightNode, rightDepth = self.dfs(root.right)
        if leftDepth > rightDepth :
            return leftNode, leftDepth + 1
        elif rightDepth > leftDepth :
            return rightNode, rightDepth + 1
        return root, leftDepth + 1
    def lcaDeepestLeaves( self, root : Optional[TreeNode] ) -> Optional[TreeNode] :
        node, _ = self.dfs(root)
        return node
```

T30102:完美交易窗口

总时间限制：1000ms 内存限制：131072kB

描述

作为一名量化交易员，你正在分析一只股票连续 N 秒内的价格数据，记录为一个数组 $h = [h_1, h_2, \dots, h_N]$ 。根据你的三十年量化经验，你定义了一种“完美交易窗口”，并且想找出这组数据中最长的完美交易窗口有多少秒。具体定义如下：

- 完美买入：必须在第 i 秒买入，且 $h[i]$ 必须是 $[i \dots j]$ 这个时间窗口内的最低价格。
- 完美卖出：必须在第 j 秒卖出，且 $h[j]$ 必须是 $[i \dots j]$ 这个时间窗口内的最高价格。
- 盈利交易：卖出价 $h[j]$ 必须严格大于买入价 $h[i]$ （即 $h[j] > h[i]$ ）。
- 严格完美：在持有期间（即第 k 秒， $i < k < j$ ），股价既不能等于你的买入价 $h[i]$ ，也不能等于你的卖出价 $h[j]$ 。

输入

第一行包含一个整数 N ($2 \leq N \leq 1,000,000$)，代表数据的持续时间（以秒为单位）。
接下来 N 行，每行包含一个整数 h_i ($1 \leq h_i < 2^{31}$)，代表第 i 秒的股票价格。

输出

一行，包含一个整数，表示最长的“完美交易窗口”有多少秒。

样例输入

```
10
3
1
2
5
4
6
1
2
4
3
```

样例输出

```
5
```

提示

tag: monotonic stack, binary search


```

import sys
from bisect import bisect_right

def solve():
    # 使用 sys.stdin.read().split() 处理百万级数据最快
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    h = list(map(int, input_data[1:]))

    # max_stk: 维护左侧第一个 >= h[j] 的位置
    max_stk = []
    # buy_stk: 存储所有可能的买入点 i 的下标 (价格严格递增)
    buy_stk = []

    # 性能优化: 缓存方法
    m_pop = max_stk.pop
    m_append = max_stk.append
    b_pop = buy_stk.pop
    b_append = buy_stk.append

    res = 0

    for j in range(n):
        cur_p = h[j]

        # 逻辑 1: 寻找左侧第一个阻碍 h[j] 成为最大值的“大数”
        while max_stk and h[max_stk[-1]] < cur_p:
            m_pop()
        # left_barrier 之前的下标都不能作为买入点 i
        left_barrier = max_stk[-1] if max_stk else -1

        # 逻辑 2: 维护买入点候选集
        # 如果当前卖出价 <= 某个买入价, 该买入点失效 (违反严格完美或盈利条件)
        while buy_stk and h[buy_stk[-1]] >= cur_p:
            b_pop()

        # 逻辑 3: 在候选买入点中, 通过二分查找躲开左侧的“阻碍大数”
        if buy_stk:
            # 找到 buy_stk 中第一个大于 left_barrier 的下标位置
            idx = bisect_right(buy_stk, left_barrier)

```

```

        if idx < len(buy_stk):
            best_i = buy_stk[idx]
            # 计算当前完美窗口长度
            res = max(res, j - best_i + 1)

# 将当前下标入栈
m_append(j)
b_append(j)

print(res)

if __name__ == "__main__":
    solve()

```

M30637:合法出栈序列pub

Description

给定一个由大小写字母和数字构成的，没有重复字符的长度不超过62的字符串x，现在要将该字符串的字符依次压入栈中要求左边的字符一定比右边的字符先入栈，出栈顺序无要求。

再给定若干字符串，对每个字符串，判断其是否是可能的x中的字符的出栈序列。

Input

第一行是原始字符串x

后面有若干行(不超过50行)，每行一个字符串，所有字符串长度不超过100

Output

对除第一行以外的每个字符串，判断其是否是可能的出栈序列。如果是，输出"YES"，否则，输出"NO"

Sample Input

```

abc
abc
bca
cab

```

Sample Output

```

YES
YES
NO

```

```

def is_valid_pop(original, target):
    # 长度不一样直接不合法
    if len(original) != len(target):
        return False
    stack = []
    push_idx = 0 # 下一个要入栈的字符索引
    for char in target:
        # 先把需要的字符入栈，直到栈顶是当前要匹配的字符
        while push_idx < len(original) and (not stack or stack[-1] != char):
            stack.append(original[push_idx])
            push_idx += 1
        # 栈顶不是目标字符，说明无法弹出
        if not stack or stack[-1] != char:
            return False
        # 弹出栈顶
        stack.pop()
    return True

# 读取输入
x = input().strip()
# 循环读取后续测试用例
while True:
    try:
        test = input().strip()
        if is_valid_pop(x, test):
            print("YES")
        else:
            print("NO")
    except EOFError:
        break

```

M04077:出栈序列统计

Description

栈是常用的一种数据结构，有 n 个元素在栈顶端一侧等待进栈，栈顶端另一侧是出栈序列。你已经知道栈的操作有两种：

Input

就一个数 n ($1 \leq n \leq 15$)。

Output

一个数，即可能输出序列的总数目。

Sample Input

3

Sample Output

5

法一

合法出栈序列 $\Leftrightarrow$ 任意时刻 pop 次数 $\leq$ push 次数

答案是卡特兰数

法二

```
def generate_valid_sequences(n):
    res = []
    def dfs(push, pop, stack, seq):
        if pop == n:
            res.append(seq.copy())
            return
        # 可以入栈
        if push < n:
            stack.append(push + 1)
            dfs(push + 1, pop, stack, seq)
            stack.pop()
        # 可以出栈
        if pop < push:
            x = stack.pop()
            seq.append(x)
            dfs(push, pop + 1, stack, seq)
            seq.pop()
            stack.append(x)
    dfs(0, 0, [], [])
    return res
```

20140:今日化学论文

Description

常凯申同学发现自己今日化学论文字数抄上限了，决定采取如下的压缩方法萌混过关：

把连续的x个字符串s记为[xs]。(1 <= x <= 100, s不包含数字)

但这样的方法当然骗不过lwh老师啦。老师非常生气，但出于好奇，还是想看一看常凯申同学写了什么。

请你帮老师还原出原始的论文。

Input

仅一行，由小写英文字母、数字和[]组成的字符串（其中不含空格）

Output

一行，原始的字符串。

Sample Input

[2b[3a]c]

Sample Output

baaacbaaac

法一：递归

```
mystring = input()
def dfs(mystring,i):
    stack = []
    num = 0
    while i < len(mystring):
        a = mystring[i]
        if a.isdigit():
            num = num*10+int(a)
        elif a == "[":
            newstack,newindex = dfs(mystring,i+1)
            stack.extend(newstack)
            i = newindex
        elif a == "]":
            return stack*num,i
        else:
            stack.append(a)
        i += 1
    return stack,i
mylist,i = dfs(mystring,0)
print(''.join(mylist))
```

法二：栈

```

def decode_string(s):
    stack = [] # 栈元素: (上层字符串, 当前层重复次数)
    current_str = ""
    current_num = 0
    for char in s:
        if char.isdigit(): # 处理多位数 (题目1-100, 兼容)
            current_num = current_num * 10 + int(char)
        elif char == '[': # 入栈: 保存当前状态
            stack.append((current_str, current_num))
            current_str = "" # 重置内层字符串
            current_num = 0 # 重置内层数字
        elif char == ']': # 出栈: 解压当前层
            prev_str, num = stack.pop()
            current_str = prev_str + current_str * num
        else: # 普通字母
            current_str += char
    return current_str

# 输入输出
inp = input().strip()
print(decode_string(inp))

```

02406:字符串乘方

总Time Limit: 3000ms Memory Limit: 65536kB

Description

给定两个字符串a和b,我们定义a*b为他们的连接。例如, 如果a="abc" 而b="def", 则a*b="abcdef"。 如果我们

Input

每一个测试样例是一行可打印的字符作为输入, 用s表示。s的长度至少为1, 且不会超过一百万。最后的测试样例后面将

Output

对于每一个s, 你应该打印最大的n, 使得存在一个a, 让 $s=a^n$

Sample Input

abcd

aaaa

ababab

.

Sample Output

1

4

3

```
import sys
data = iter(sys.stdin.readlines())
def kmp(s):
    j = 0
    n = len(s)-1
    lps = [0]*n
    for i in range(1,n):
        while j > 0 and s[j] != s[i]:
            j = lps[j-1]
        if s[j] == s[i]:
            j += 1
        lps[i] = j
    jie = n-lps[n-1]
    if n%jie == 0:
        return n//jie
    return 1
while True:
    s = next(data)
    if s[0] == ".":
        break
    print(kmp(s))
```

30178:数字华容道（Easy Version）

总时间限制：2000ms 单个测试点时间限制：1000ms 内存限制：262144kB

描述

数字华容道的游戏玩法：在一个 $n * n$ 的方格中放置 1 到 n^2-1 的所有数字（每个数字占据一空间），剩下一个空格。

```
1 2 3 4
5 6 7 8
9 10 11 12
15 13 14 0
```

你的目标是通过平移数字，将盘面变成

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

的样子。一天，小K正准备玩数字华容道，但是他不小心把盘面中的数字小方格都打翻在地了。虽然他将所有的数字都

输入

第 1 行：一个整数 n ，($2 \leq n \leq 1000$)。（你就不要问小K是怎么把一百万个小方格捡起来的了）

第 2 行到第 $n+1$ 行： n 个整数，表示盘面。

输出

如果盘面有解，输出yes，否则输出no

样例输入

```
4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

样例输出

yes

当 n 为偶数时：逆序数 + 空格行号（从下往上数）为奇数 → 有解

当 n 为奇数时：仅需逆序数为偶数 → 有解


```

n = int(input())
sequence = []
for i in range(n):
    mylist = list(map(int, input().split()))
    if 0 in mylist:
        s = n-i+1
        mylist.remove(0)
    sequence.extend(mylist)
shu = [0 for _ in range(n**2)]
def update(i):
    while i < len(shu):
        shu[i] += 1
        i += (i&(-i))
def query(t):
    total = 0
    while t > 0:
        total += shu[t]
        t -= (t&(-t))
    return total
count = 0
for num in sequence:
    num = n**2-num
    count += (query(num)%2)
    count %= 2
    update(num)
if n % 2 == 1:
    if not count:
        print("yes")
    else:
        print("no")
else:
    if (count + s) % 2 == 0:
        print("yes")
    else:
        print("no")

```

437 Path Sum III

Given the root of a binary tree and an integer targetSum, return the number of paths where the path does not need to start or end at the root or a leaf, but it must go downwards.

Example 1:

Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8

Output: 3

Explanation: The paths that sum to 8 are shown.

Example 2:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

Output: 3

Constraints:

The number of nodes in the tree is in the range [0, 1000].

$-109 \leq \text{Node.val} \leq 109$

$-1000 \leq \text{targetSum} \leq 1000$

这个思路很好，把要存储的东西跟着dfs走

```
from collections import defaultdict
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        prefix_sum = defaultdict(int)
        prefix_sum[0] = 1

        def dfs(node, curr_sum):
            if not node:
                return 0

            curr_sum += node.val
            count = prefix_sum[curr_sum - targetSum]

            prefix_sum[curr_sum] += 1

            count += dfs(node.left, curr_sum)
            count += dfs(node.right, curr_sum)

            prefix_sum[curr_sum] -= 1
            return count

        return dfs(root, 0)
```

37 Sudoku Solver

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

Each of the digits 1–9 must occur exactly once in each row.

Each of the digits 1–9 must occur exactly once in each column.

Each of the digits 1–9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8",".","3",".","."],
["7",".",".","2",".","."],
[".","6",".",".","2","8","."],
[".","4","1","9",".","5"],
[".",".","8",".","7","9"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],
["6","7","2","1","9","5","3","4","8"],
["1","9","8","3","4","2","5","6","7"],
["8","5","9","7","6","1","4","2","3"],
["4","2","6","8","5","3","7","9","1"],
["7","1","3","9","2","4","8","5","6"],
["9","6","1","5","3","7","2","8","4"],
["2","8","7","4","1","9","6","3","5"],
["3","4","5","2","8","6","1","7","9"]]
```

Explanation: The input board is shown above and the only valid solution is shown below

这道题像是做项目一样

首先backtrack毫无争议

无非思考的是三个问题：更新什么，怎么填数，怎么撤回

最开始我以为更新的是每个格的可能情形，用一个二进制数存起来

但是这样每次查找一个格的可能情形，就要花20下

填数不能乱填，那种有很多可能的格不好，宁愿花点时间去算最小可能的格也是值得的

撤回的时候一定要恢复，如若不恢复，访问每一格都要重复更新

用原来更新思路就会导致填数每一次要花1600下左右

更新策略有问题，不应该去每次把邻居都跑一遍，应该用一个东西来存整体的状态

那就是用行列块来存储，这样每次查找一个格只需要3下

```

class Solution:
    def solveSudoku(self, board) -> None:
        hang, lie, kuai = [0 for _ in range(9)], [0 for _ in range(9)], [0 for _ in range(9)]
        vac = set()
        for i in range(9):
            for j in range(9):
                if board[i][j] == ".":
                    vac.add((i, j))
        def update(i, j):
            num = 0
            num = num | hang[i] | lie[j] | kuai[3*(i//3)+(j//3)]
            return num
        def findmin():
            i1, j1 = 10, 10
            maxnum = -1
            for i, j in vac:
                a = update(i, j)
                if a > maxnum:
                    i1, j1 = i, j
                    maxnum = a
            return i1, j1
        def uplin(i, j):
            a = int(board[i][j])
            hang[i] = hang[i] | (1<<a)
            lie[j] = lie[j] | (1<<a)
            kuai[3*(i//3)+(j//3)] = kuai[3*(i//3)+(j//3)] | (1<<a)
        def downlin(i, j):
            a = int(board[i][j])
            a = 1022-(1<<a)
            hang[i] = hang[i]&a
            lie[j] = lie[j]&a
            kuai[3*(i//3)+(j//3)] = kuai[3*(i//3)+(j//3)]&a
        for i in range(9):
            for j in range(9):
                if (i, j) not in vac:
                    uplin(i, j)
        i, j = findmin()
        def dfs(i, j):
            if i == 10:
                return True
            m = update(i, j)
            for s in range(1, 10):
                t = (1<<s) | m

```

```

        if t == m:
            continue
        board[i][j] = str(s)
        uplin(i,j)
        vac.discard((i,j))
        i1,j1 = findmin()
        if dfs(i1,j1):
            return True
        downlin(i,j)
        board[i][j] = "."
        vac.add((i,j))
    return False
dfs(i,j)

```

787 Cheapest Flights Within K Stops

There are n cities connected by some number of flights. You are given an array `flights`. You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops.

Example 1:

Input: $n = 4$, `flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

Output: 700

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost 1000.

Note that the path through cities `[0,1,2,3]` is cheaper but is invalid because it uses 2 stops.

Example 2:

Input: $n = 3$, `flights = [[0,1,100],[1,2,100],[0,2,500]]`, `src = 0`, `dst = 2`, `k = 1`

Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost 1000.

Example 3:

Input: $n = 3$, `flights = [[0,1,100],[1,2,100],[0,2,500]]`, `src = 0`, `dst = 2`, `k = 0`

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

必须走 $k+1$ 步的bfs，然后每一步先探测可以到哪些节点，然后再一起更新，边走边更新会增加步长

```

class Solution:
    def findCheapestPrice(self, n: int, flights, src: int, dst: int, k: int) -> int:
        from collections import defaultdict
        linjiebiao = defaultdict(list)
        for loc1, loc2, price in flights:
            linjiebiao[loc1].append((loc2, price))
        dists = [float("inf") for _ in range(n)]
        dists[src] = 0
        queue = {src}
        for _ in range(k+1):
            newset = set()
            cun = {}
            while queue:
                loc = queue.pop()
                dis = dists[loc]
                for loc3, dis3 in linjiebiao[loc]:
                    newdis = dis3+dis
                    if newdis < dists[loc3]:
                        cun[loc3] = min(newdis, cun.get(loc3, float("inf")))
                        newset.add(loc3)
            for loc in newset:
                dists[loc] = cun[loc]
            queue = newset
        if dists[dst] != float("inf"):
            return dists[dst]
        return -1

```

1829E The Lakes

time limit per test 3 seconds

memory limit per test 256 megabytes

You are given an $n \times m$ grid a of non-negative integers. The value $a_{i,j}$ represents the depth

A lake is a set of cells such that:

each cell in the set has $a_{i,j} > 0$, and

there exists a path between any pair of cells in the lake by going up, down, left, or right.

The volume of a lake is the sum of depths of all the cells in the lake.

Find the largest volume of a lake in the grid.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) – the number of test cases.

The first line of each test case contains two integers n, m ($1 \leq n, m \leq 1000$) – the number of rows and columns.

Then n lines follow each with m integers $a_{i,j}$ ($0 \leq a_{i,j} \leq 1000$) – the depth of the water at cell (i, j) .

It is guaranteed that the sum of $n \cdot m$ over all test cases does not exceed 10^6 .

Output

For each test case, output a single integer – the largest volume of a lake in the grid.

Example

Input

```
5
3 3
1 2 0
3 4 0
0 0 5
1 1
0
3 3
0 1 1
1 0 1
1 1 1
5 5
1 1 1 1 1
1 0 0 0 1
1 0 5 0 1
1 0 0 0 1
1 1 1 1 1
5 5
1 1 1 1 1
1 0 0 0 1
1 1 4 0 1
1 0 0 0 1
1 1 1 1 1
```

Output

10
0
7
16
21

如果dfs爆栈，那么就用stack写法

```
import sys
data = sys.stdin.read().split()
case = int(data[0])
t = 1
for _ in range(case):
    m,n = int(data[t]),int(data[t+1])
    t += 2
    grid = []
    for __ in range(m):
        grid.append([int(data[j]) for j in range(t,t+n)])
        t += n
    visited = [[0]*n for s in range(m)]
    vectors = [(0,1),(0,-1),(-1,0),(1,0)]
    maximum = 0
    for i in range(m):
        for j in range(n):
            if visited[i][j] == 0 and grid[i][j] != 0:
                stack = [(i,j)]
                total = 0
                visited[i][j] = 1
                while stack:
                    i1,j1 = stack.pop()
                    total += grid[i1][j1]
                    for a,b in vectors:
                        if i1+a >= 0 and i1+a <= m-1 and j1+b >= 0 and j1+b <= n-1 and
                            stack.append((i1+a,j1+b))
                            visited[i1+a][j1+b] = 1
                maximum = max(maximum,total)
    print(maximum)
```


28050:骑士周游

总Time Limit: 1000ms Memory Limit: 65536kB

Description

在一个国际象棋棋盘上，一个棋子“马”（骑士），按照“马走日”的规则，从一个格子出发，要走遍所有棋盘格恰好一次。采用图搜索算法，是解决骑士周游问题最容易理解和编程的方案之一，解决方案分为两步： 首先用图表示骑士在棋盘上

Input

两行。

第一行是一个整数 n ，表示正方形棋盘边长， $3 \leq n \leq 19$ 。

第二行是空格分隔的两个整数 sr, sc ，表示骑士的起始位置坐标。棋盘左上角坐标是 $0\ 0$ 。 $0 \leq sr \leq n-1, 0 \leq sc \leq n-1$

Output

如果是合格的周游，输出 `success`，否则输出 `fail`。

Sample Input

```
5
0 0
```

Sample Output

```
success
```

选择下一个要访问的顶点至关重要

Warnsdorff算法:选择合理走法最多的顶点作为下一个访问顶点的问题在于，它会使骑士在周游的前期就访问位于棋盘中间的格子。当这种情况发生时，骑士很容易被困在棋盘的一边，而无法到达另一边的那些没访问过的格子。首先访问合理走法最少的顶点，则可使骑士优先访问棋盘边缘的格子。这样做保证了骑士能够尽早访问难以到达的角落，并且在需要的时候通过中间的格子跨越到棋盘的另一边。

```

n = int(input())
x,y = map(int,input().split())
vector = [(1,2),(1,-2),(-1,2),(-1,-2),(2,1),(2,-1),(-2,1),(-2,-1)]
def available(lc):
    mylist = []
    for myvector in vector:
        a,b = lc[0]+myvector[0],lc[1]+myvector[1]
        if a >= 0 and a <= n-1 and b >= 0 and b <= n-1:
            mylist.append((a,b))
    return mylist
def count(lc,myset):
    count = 0
    for t in available(lc):
        if t not in myset:
            count += 1
    return count
def choose(mylist,myset):
    mylist.sort(key=lambda x:count(x,myset))
    return mylist
def backtrack(lc,myset):
    if len(myset) == n*n:
        return True
    mylist = choose(available(lc),myset)
    for lc1 in mylist:
        if lc1 not in myset:
            myset.add(lc1)
            a = backtrack(lc1,myset)
            if a:
                myset.discard(lc1)
                return True
            myset.discard(lc1)
    return False
if backtrack((x,y),{(x,y)}):
    print("success")
else:
    print("fail")

```

二叉树的中序遍历 (bfs版)

```
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        white, gray = 0, 1
        res = []
        stack = [(white, root)]
        while stack:
            color, node = stack.pop()
            if node is None: continue
            if color == white:
                stack.append((white, node.right))
                stack.append((gray, node))
                stack.append((white, node.left))
            else:
                res.append(node.val)
        return res
```

127 Word Ladder

A transformation sequence from word beginWord to word endWord using a dictionary wordList. Every adjacent pair of words differs by a single letter.

Every s_i for $1 \leq i \leq k$ is in wordList. Note that beginWord does not need to be in wordList. $s_k == endWord$

Given two words, beginWord and endWord, and a dictionary wordList, return the number of words in the shortest transformation sequence from beginWord to endWord.

Example 1:
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5

Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog".
Example 2:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: 0

Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.
Constraints:

$1 \leq \text{beginWord.length} \leq 10$

$\text{endWord.length} == \text{beginWord.length}$

$1 \leq \text{wordList.length} \leq 5000$

$\text{wordList}[i].\text{length} == \text{beginWord.length}$

beginWord, endWord, and wordList[i] consist of lowercase English letters.

beginWord != endWord

All the words in wordList are unique.

```

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        wordset = set(wordList)
        if endWord not in wordset:
            return 0
        beginSet = {beginWord}
        endSet = {endWord}
        visited = {beginWord, endWord}
        step = 1
        n = len(beginWord)
        while beginSet and endSet:
            if len(beginSet) > len(endSet):
                beginSet, endSet = endSet, beginSet
            nextSet = set()
            for word in beginSet:
                word_list = list(word)
                for i in range(len(word)):
                    origin = word_list[i]
                    for c in "abcdefghijklmnopqrstuvwxyz":
                        if c == origin:
                            continue
                        word_list[i] = c
                        newWord = "".join(word_list)
                        if newWord in endSet:
                            return step + 1
                        if newWord in wordset and newWord not in visited:
                            nextSet.add(newWord)
                            visited.add(newWord)
                    word_list[i] = origin
            beginSet = nextSet
            step += 1
        return 0

```

01182:食物链

总Time Limit: 1000ms Memory Limit: 65536kB

Description

动物王国中有三类动物A,B,C,这三类动物的食物链构成了有趣的环形。A吃B, B吃C, C吃A。

现有N个动物,以1-N编号。每个动物都是A,B,C中的一种,但是我们并不知道它到底是哪一种。

有人用两种说法对这N个动物所构成的食物链关系进行描述:

第一种说法是"1 X Y",表示X和Y是同类。

第二种说法是"2 X Y",表示X吃Y。

此人对N个动物,用上述两种说法,一句接一句地说出K句话,这K句话有的是真的,有的是假的。当一句话满足下列三条

- 1) 当前的话与前面的某些真的话冲突,就是假话;
- 2) 当前的话中X或Y比N大,就是假话;
- 3) 当前的话表示X吃X,就是假话。

你的任务是根据给定的N ($1 \leq N \leq 50,000$) 和K句话 ($0 \leq K \leq 100,000$), 输出假话的总数。

Input

第一行是两个整数N和K,以一个空格分隔。

以下K行每行是三个正整数 D, X, Y, 两数之间用一个空格隔开,其中D表示说法的种类。

若D=1,则表示X和Y是同类。

若D=2,则表示X吃Y。

Output

只有一个整数,表示假话的数目。

Sample Input

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

Sample Output

```
3
```

type[x] % 3 ==

0 : x 和 x的根结点 同类

1 : x 吃 x的根结点

2 : x的根结点 吃 x

i → i1 (i的根): type[i] = (i与i1是否同类的数)

```

N,K = map(int,input().split())
Parent = [i for i in range(N)]
type = [0]*N
count = 0
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        type[i] = (type[i]+type[Parent[i]])%3 #这里是找以前的父节点，后面率先更新的type应该是以
        Parent[i] = result
        return result
for _ in range(K):
    D,i,j = map(int,input().split())
    if i >= N+1 or i <= 0 or j >= N+1 or j <= 0:
        count += 1
        continue
    i -= 1
    j -= 1
    if D == 1:
        i1 = find(i)
        j1 = find(j)
        if i1 == j1:
            if (type[i] - type[j]) % 3 != 0:
                count += 1
                continue
        else:
            Parent[j1] = i1
            type[j1] = (type[i]-type[j])%3
    if D == 2:
        i1 = find(i)
        j1 = find(j)
        if i1 == j1:
            if (type[i]-type[j]-1)%3 != 0:
                count += 1
                continue
        else:
            Parent[i1] = j1
            type[i1] = (type[j]-type[i]+1)%3
print(count)

```

kd树

找二维平面的离原点最近的k近邻

```

from typing import List
import heapq

class TreeNode:
    def __init__(self, pos):
        self.pos = pos #[x,y]
        self.left = None
        self.right = None
        self.split = 0 #拆分维度, 0表示拆x, 1表示拆y

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:

        def construct(dim, points):
            '''build a kd-tree, return the current root node'''
            if not points:
                return None
            points.sort(key=lambda x: x[dim])
            mid = len(points)//2
            node = TreeNode(points[mid])
            node.split = dim
            node.left = construct((dim+1)%2, points[:mid])
            node.right = construct((dim+1)%2, points[(mid+1):])
            return node

        root = construct(0, points)

        heap = [] #每次弹最大的
        curmax = 0

        def dis(pos):
            return pos[0]*pos[0]+pos[1]*pos[1]

        def search(node):
            '''search a kd-tree and return None'''
            nonlocal curmax

            if not node:
                return

            if node.pos[node.split] > 0:
                search(node.left)
            else:

```



```
search(node.right)
```

```
if len(heap) < k:  
    heapq.heappush(heap, (-dis(node.pos), node.pos))  
    curmax = max(-heap[0][0], curmax)  
elif len(heap) == k:  
    distance = dis(node.pos)  
    if distance < curmax:  
        heapq.heappushpop(heap, (-dis(node.pos), node.pos))  
        curmax = max(-heap[0][0], curmax)  
    if node.pos[node.split]**2 >= curmax:  
        return  
  
if node.pos[node.split] > 0:  
    search(node.right)  
else:  
    search(node.left)
```

```
search(root)
```

```
return [pair[1] for pair in heap]
```

```
print(Solution().kClosest([[1,3], [-2,2], [2,-2]],2))
```

01145:Tree Summing

Description

LISP was one of the earliest high-level programming languages and, with FORTRAN, is one of the most widely used. This problem deals with determining whether binary trees represented as LISP S-expressions are summing trees. Given a binary tree of integers, you are to write a program that determines whether the sum of the root's two children is equal to the root's value. Binary trees are represented in the input file as LISP S-expressions having the following form:

empty tree ::= ()

tree ::= empty tree (integer tree tree)

The tree diagrammed above is represented by the expression (5 (4 (11 (7 () ())) (2 () ()))

Note that with this formulation all leaves of a tree are of the form (integer () ())

Since an empty tree has no root-to-leaf paths, any query as to whether a path exists will return false.

Input

The input consists of a sequence of test cases in the form of integer/tree pairs. Each test case is on a single line.

Output

There should be one line of output for each test case (integer/tree pair) in the input. The output for each test case is "yes" if the tree is a summing tree, and "no" otherwise.

Sample Input

```
22 (5(4(11(7())(2())))) (8(13())(4()(1()))))
```

```
20 (5(4(11(7())(2())))) (8(13())(4()(1()))))
```

```
10 (3
```

```
    (2 (4 () ())
```

```
        (8 () ()) ) )
```

```
    (1 (6 () ())
```

```
        (4 () ()) ) ) )
```

```
5 ()
```

Sample Output

```
yes
```

```
no
```

```
yes
```

```
no
```

sys读入会读入“\n”，所以需要先处理一下整理成好看的data列表形式

接着建树，思路就是global的self.index去利用前序遍历

注意整数可能是负数，要小心负号的处理

```

import sys
rawdata = sys.stdin.read()
data = []
s1 = ""
for s in rawdata:
    if s.isdigit() or s == "-":
        s1 += s
    elif s == " " or s == "\n":
        continue
    else:
        if s1:
            data.append(s1)
            s1 = ""
        data.append(s)

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def solve(mylist, num):
    class Tree:
        def __init__(self):
            self.tree = None
            self.index = 0
            self.count = 0

        def construct(self, node):
            while True:
                a = mylist[self.index]
                self.index += 1
                if a == "(":
                    continue
                if a == ")":
                    return node
                node = TreeNode(int(a))
                node.left = self.construct(node.left)
                node.right = self.construct(node.right)

        def build(self):
            self.tree = self.construct(self.tree)

    tree = Tree()
    tree.build()
    t = tree.tree

    def preorder_traversal(node, cursum):
        cursum += node.val

```

```

    if not node.left and not node.right:
        if cursum == num:
            return True
        return False
    if node.left:
        a = preorder_traversal(node.left, cursum)
        if a:
            return True
    if node.right:
        b = preorder_traversal(node.right, cursum)
        if b:
            return True
    return False
if preorder_traversal(t, 0):
    print("yes")
else:
    print("no")

i = 0
count = 0
while i < len(data):
    if count == 0 and (data[i][0].isdigit() or data[i][0] == "-"):
        num = int(data[i])
        i += 1
        j = i
        continue
    a = data[i]
    i += 1
    if a == "(":
        count += 1
    elif a == ")":
        count -= 1
    if count == 0:
        mylist = data[j:i]
        if mylist == ["(", ")"]:
            print("no")
        else:
            solve(mylist, num)

```

20018:蚂蚁王国的越野跑

总Time Limit: 1000ms Memory Limit: 262144kB

Description

为了促进蚂蚁家族身体健康，提高蚁族健身意识，蚂蚁王国举行了越野跑。假设越野跑共有N个蚂蚁参加，在一条笔直的

Input

第一行1个整数N。

第2... N +1行: N 个非负整数，按从前到后的顺序给出每个蚂蚁的跑步速度。对于50%的数据， $2 \leq N \leq 1000$ 。对于10

Output

一个整数，表示有多少对参赛者之间发生赶超事件。

Sample Input

5
1
5
10
7
6

5
1
5
5
7
6

Sample Output

7

8

Hint

我们把这5个蚂蚁依次编号为A,B,C,D,E，假设速度分别为1,5,5,7,6。在跑步过程中：B,C,D,E均会超过A，因为他1

```

class BIT:
    def __init__(self,size):
        self.size = size
        self.tree = [0]*size
    def lowbit(self,x):
        return x&(-x)
    def insert(self,i):
        a = self.query(i-1)
        while i <= self.size:
            self.tree[i-1] += 1
            i += self.lowbit(i)
        return a
    def query(self,i):
        total = 0
        while i:
            total += self.tree[i-1]
            i -= self.lowbit(i)
        return total

N = int(input())
mylist = []
for _ in range(N):
    mylist.append(int(input()))
bit = BIT(max(mylist))
total1 = 0
for num in mylist:
    total1 += bit.insert(num)
print(total1)

```

AVL树的插入节点与删除节点

删除节点是把右子树最小的节点拿来补位

avl插入（ll型根右旋，lr型先左子树左旋再根右旋，rr型根左旋，rl型先右子树右旋再根左旋）

```

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
        self.height = 1

class AVL:
    def __init__(self):
        self.root = None

    def insert(self, value):
        if not self.root:
            self.root = Node(value)
        else:
            self.root = self._insert(value, self.root)

    def _insert(self, value, node):
        if not node:
            return Node(value)
        elif value < node.value:
            node.left = self._insert(value, node.left)
        else:
            node.right = self._insert(value, node.right)
        node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))
        balance = self._get_balance(node)
        if balance > 1:
            if value < node.left.value:          # 树形是 LL
                return self._rotate_right(node)
            else: # 树形是 LR
                node.left = self._rotate_left(node.left)
                return self._rotate_right(node)
        if balance < -1:
            if value > node.right.value:         # 树形是 RR
                return self._rotate_left(node)
            else: # 树形是 RL
                node.right = self._rotate_right(node.right)
                return self._rotate_left(node)
        return node

    def _get_height(self, node):
        if not node:
            return 0
        return node.height

    def _get_balance(self, node):
        if not node:
            return 0

```

```

        return self._get_height(node.left) - self._get_height(node.right)
def _rotate_left(self, z):
    y = z.right
    T2 = y.left
    y.left = z
    z.right = T2
    z.height = 1 + max(self._get_height(z.left), self._get_height(z.right))
    y.height = 1 + max(self._get_height(y.left), self._get_height(y.right))
    return y
def _rotate_right(self, y):
    x = y.left
    T2 = x.right
    x.right = y
    y.left = T2
    y.height = 1 + max(self._get_height(y.left), self._get_height(y.right))
    x.height = 1 + max(self._get_height(x.left), self._get_height(x.right))
    return x
def delete(self, value):
    self.root = self._delete(value, self.root)
def _delete(self, value, node):
    if not node:
        return node
    if value < node.value:
        node.left = self._delete(value, node.left)
    elif value > node.value:
        node.right = self._delete(value, node.right)
    else:
        if not node.left:
            temp = node.right
            node = None
            return temp
        elif not node.right:
            temp = node.left
            node = None
            return temp
        temp = self._min_value_node(node.right)
        node.value = temp.value
        node.right = self._delete(temp.value, node.right)
    node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))
    balance = self._get_balance(node)
    # Rebalance the tree
    if balance > 1:
        if self._get_balance(node.left) >= 0:

```



```
        return self._rotate_right(node)
    else:
        node.left = self._rotate_left(node.left)
        return self._rotate_right(node)
if balance < -1:
    if self._get_balance(node.right) <= 0:
        return self._rotate_left(node)
    else:
        node.right = self._rotate_right(node.right)
        return self._rotate_left(node)
return node
def _min_value_node(self, node):
    current = node
    while current.left:
        current = current.left
    return current
```

堆排序

利用最大堆的性质，将列表从小到大排序。

```

class Node:
    def __init__(self, key):
        self.key = key
# 确保以索引 i 为根节点的子树满足最大堆的性质（父节点的值大于等于子节点的值）。
def adjust_heap(lst, i, size):
    lchild = 2 * i + 1
    rchild = 2 * i + 2
    max_idx = i
    if i < size // 2:
        if lchild < size and lst[lchild].key > lst[max_idx].key:
            max_idx = lchild
        if rchild < size and lst[rchild].key > lst[max_idx].key:
            max_idx = rchild
        if max_idx != i:
            lst[max_idx], lst[i] = lst[i], lst[max_idx]
            adjust_heap(lst, max_idx, size)
def build_heap(lst):
    size = len(lst)
    for i in range(0, (size // 2))[::-1]: # 从最后一个非叶子节点开始，逐个向上调整，使整个列表成
        adjust_heap(lst, i, size)
def heap_sort(lst):
    size = len(lst)
    build_heap(lst)
    for i in range(0, size)[::-1]:
        lst[0], lst[i] = lst[i], lst[0]
        adjust_heap(lst, 0, i)
def print_list(lst):
    return [node.key for node in lst]
if __name__ == "__main__":
    lst = [Node(5), Node(9), Node(3), Node(8), Node(7)]
    # 打印排序前的列表
    print("Before sorting:", print_list(lst))
    # 调用堆排序
    heap_sort(lst)
    # 打印排序后的列表
    print("After sorting:", print_list(lst))
"""
Before sorting: [5, 9, 3, 8, 7]
After sorting: [3, 5, 7, 8, 9]
"""

```

04078:实现堆结构

Description

定义一个数组，初始化为空。在数组上执行两种操作：

- 1、增添1个元素，把1个新的元素放入数组。
- 2、输出并删除数组中最小的数。

使用堆结构实现上述功能的高效算法。

Input

第一行输入一个整数 n ，代表操作的次数。

每次操作首先输入一个整数 $type$ 。

当 $type=1$ ，增添操作，接着输入一个整数 u ，代表要插入的元素。

当 $type=2$ ，输出删除操作，输出并删除数组中最小的元素。

$1 \leq n \leq 100000$ 。

Output

每次删除操作输出被删除的数字。

Sample Input

```
4
1 5
1 1
1 7
2
```

Sample Output

```
1
```

Hint

每组测试数据的复杂度为 $O(n \log n)$ 的算法才能通过本次，否则会返回TLE(超时)

需要使用最小堆结构来实现本题的算法

```

import sys
data = iter(sys.stdin.read().split())
class Binaryheap:
    def __init__(self):
        self.heap = []
    def insert(self,i):
        self.heap.append(i)
        cur = len(self.heap)
        while cur != 1:
            cur1 = cur//2
            if self.heap[cur1-1] > i:
                self.heap[cur1-1],self.heap[cur-1] = self.heap[cur-1],self.heap[cur1-1]
                cur = cur1
            else:
                break
    def exchange(self,i):
        if not self.heap:
            return
        m = self.heap[i-1]
        #把最小的换上去
        if 2*i+1 <= len(self.heap) and m > min(self.heap[2*i],self.heap[2*i-1]):
            if self.heap[2*i] <= self.heap[2*i-1]:
                self.heap[i-1],self.heap[2*i] = self.heap[2*i],self.heap[i-1]
                self.exchange(2*i+1)
            else:
                self.heap[i-1],self.heap[2*i-1] = self.heap[2*i-1],self.heap[i-1]
                self.exchange(2*i)
        elif 2*i == len(self.heap) and m > self.heap[2*i-1]:
            self.heap[i-1],self.heap[2*i-1] = self.heap[2*i-1],self.heap[i-1]
            self.exchange(2*i)
    def delete(self):
        self.heap[0],self.heap[-1] = self.heap[-1],self.heap[0]
        a = self.heap.pop()
        self.exchange(1)
        return a
heapq = Binaryheap()
for _ in range(int(next(data))):
    a = next(data)
    if a == "1":
        b = int(next(data))
        heapq.insert(b)

```

```
else:  
    print(heapq.delete())
```

3097 Shortest Subarray With OR at Least K II

You are given an array `nums` of non-negative integers and an integer `k`.

An array is called special if the bitwise OR of all of its elements is at least `k`.

Return the length of the shortest special non-empty subarray of `nums`, or return `-1` if `n`

Example 1:

Input: `nums = [1,2,3]`, `k = 2`

Output: 1

Explanation:

The subarray `[3]` has OR value of 3. Hence, we return 1.

Example 2:

Input: `nums = [2,1,8]`, `k = 10`

Output: 3

Explanation:

The subarray `[2,1,8]` has OR value of 11. Hence, we return 3.

Example 3:

Input: `nums = [1,2]`, `k = 0`

Output: 1

Explanation:

The subarray `[1]` has OR value of 1. Hence, we return 1.

Constraints:

`1 <= nums.length <= 2 * 10**5`

`0 <= nums[i] <= 10**9`

`0 <= k <= 10**9`

一般这种找最短子序列的题都可以往滑动窗口思考

亮点在于滑动窗口滑动可以带着两个变量一起滑

不要惧怕位数，位数顶多32，可以跟着一起存储

```

class Solution:
    def minimumSubarrayLength(self, nums: List[int], k: int) -> int:
        if k == 0: return 1
        ans = float('inf')
        bits = [0] * 32
        left = 0
        def update_bits(val, delta):
            for i in range(32):
                if (val >> i) & 1:
                    bits[i] += delta
        def get_or_val():
            res = 0
            for i in range(32):
                if bits[i] > 0:
                    res |= (1 << i)
            return res
        for right in range(len(nums)):
            update_bits(nums[right], 1)
            # 窗口满足条件时, 尝试收缩
            while get_or_val() >= k and left <= right:
                ans = min(ans, right - left + 1)
                update_bits(nums[left], -1)
                left += 1
        return ans if ans != float('inf') else -1

```

25140:根据后序表达式建立表达式树

Description

后序算术表达式可以通过栈来计算其值，做法就是从左到右扫描表达式，碰到操作数就入栈，碰到运算符，就取出栈顶的。有一种算术表达式不妨叫做“队列表达式”，它的求值过程和后序表达式很像，只是将栈换成了队列：从左到右扫描表达式：给定一个后序表达式，请转换成等价的队列表达式。例如，"3 4 + 6 5 * -"的等价队列表达式就是"5 6 4 3 * +

Input

第一行是正整数 n ($n < 100$)。接下来是 n 行，每行一个由字母构成的字符串，长度不超过100,表示一个后序表达式，其中

Output

对每个后序表达式，输出其等价的队列表达式。

Sample Input

```
2
xyPzwIM
abcABdefgCDEF
Sample Output
wzyxIPM
gfCecbDdAaEBF
```

建立起表达式树，按层序遍历表达式树的结果前后颠倒就得到队列表达式

体会这个思路构建流程，从小情况感受

```

import sys
data = iter(sys.stdin.read().split())
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
for _ in range(int(next(data))):
    mystr = list(next(data))
    stack = []
    for s in mystr:
        node = TreeNode(s)
        if s.isupper():
            node.right = stack.pop()
            node.left = stack.pop()
        stack.append(node)
    tree = stack[0]
    result = []
    from collections import deque
    queue = deque([tree])
    while queue:
        for _ in range(len(queue)):
            node = queue.popleft()
            result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    print(''.join(reversed(result)))

```

解析树/抽象语法树（AST）

完全括号表达式构建解析式

(1) 如果当前标记是(, 就为当前节点添加一个左子节点, 并下沉至该子节点; (2) 如果当前标记在列表 ['+', '-', '/', '*'] 中, 就将当前节点的值设为当前标记对应的运算符; 为当前节点添加一个右子节点, 并下沉至该子节点; (3) 如果当前标记是数字, 就将当前节点的值设为这个数并返回至父节点; (4) 如果当前标记是), 就跳到当前节点的父节点。

返回父节点用stack帮忙 (碰到数和')就pop)

138 Copy List with Random Pointer

A linked list of length n is given such that each node contains an additional random pointer. Construct a deep copy of the list. The deep copy should consist of exactly n brand new nodes. For example, if there are two nodes X and Y in the original list, where $X.random \rightarrow Y$, Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as: `val: an integer representing Node.val`

`random_index: the index of the node (range from 0 to  $n-1$ ) that the random pointer points to.` Your code will only be given the head of the original linked list.

Example 1:

Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]

Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

Example 2:

Input: head = [[1,1],[2,1]]

Output: [[1,1],[2,1]]

Example 3:

Input: head = [[3,null],[3,0],[3,null]]

Output: [[3,null],[3,0],[3,null]]

把每一个新复制的节点 `copy` 直接插入到原节点 `curr` 的后面。

链表变成了 $A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow C \rightarrow C'$

如果原节点 A 指向 C ，那么克隆节点 A' 的 `random` 应该指向 C'

拆解链表时要修复原链表，不能只产生克隆链表

将这个长链表拆分成两个：一个是恢复后的原链表，一个是纯粹的克隆链表

```

class Solution:
    def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
        if not head :
            return head
        temp = head
        while temp :
            curr = Node(temp.val)
            curr.next = temp.next
            temp.next = curr
            temp = curr.next
        curr = head
        while curr and curr.next :
            if curr.random :
                curr.next.random = curr.random.next
            curr = curr.next.next
        new_head = head.next
        current = head
        new_curr = new_head
        while current :
            current.next = new_curr.next
            current = current.next
            if current :
                new_curr.next = current.next
                new_curr = new_curr.next
        return new_head

```

Height of a generic tree from parent array

给定一棵大小为 n 的树，以数组 `parent[0..n-1]` 的形式表示，其中 `parent[i]` 中的每个索引 i 代表一个节点，而 i 处的值表示该节点的直接父节点。对于根节点，其值为 -1 。根据给定的父节点链接，求这棵通用树的高度。

法一：一层一层bfs

法二：我们以迭代方式访问从 0 到 $n-1$ 的所有节点，并递归地标记那些尚未被访问过的祖先节点，直到到达一个已被访问过的节点，或者到达根节点。如果在通过父节点链接向上遍历树时遇到一个已访问过的节点，那么我们会使用它的高度，并停止进一步的递归。

02299:Ultra-QuickSort

总Time Limit: 1000ms Memory Limit: 65536kB

Description

In this problem, you have to analyze a particular sorting algorithm. The algorithm proc

9 1 0 5 4 ,

Ultra-QuickSort produces the output

0 1 4 5 9 .

Your task is to determine how many swap operations Ultra-QuickSort needs to perform in

Input

The input contains several test cases. Every test case begins with a line that contains

Output

For every input sequence, your program prints a single line containing an integer numbe

Sample Input

5

9

1

0

5

4

3

1

2

3

0

Sample Output

6

0

省流：就是找逆序对个数

冒泡排序 $O(n^2)$ 会超时

必须用归并排序 $O(n \log n)$

递归深度($\log n$)

注意要原地修改mylist，否则会超内存

```

import sys
input = sys.stdin.read().split()
data = iter(input)
while True:
    n = int(next(data))
    if n == 0:
        break
    count = [0]
    mylist = [int(next(data)) for _ in range(n)]
    def mergesort(i,j):
        if i == j:
            return [mylist[i]]
        mid = (i+j)//2
        list1 = mergesort(i,mid)
        list2 = mergesort(mid+1,j)
        t,s = 0,0
        cur = i
        while cur <= j:
            if t < len(list1) and s < len(list2):
                if list1[t] > list2[s]:
                    mylist[cur] = (list2[s])
                    s += 1
                    count[0] += (s+len(list1)-(cur-i+1))
                    cur += 1
                else:
                    mylist[cur] = (list1[t])
                    t += 1
                    cur += 1
            elif t < len(list1):
                mylist[cur] = (list1[t])
                t += 1
                cur += 1
            elif s < len(list2):
                mylist[cur] = (list2[s])
                s += 1
                cur += 1
            else:
                break
        return mylist[i:j+1]
    mergesort(0,n-1)
    print(count[0])

```

02775:文件结构“图”

Description

在计算机上看到文件系统的结构通常很有用。Microsoft Windows上面的"explorer"程序就是这样的一个例子。但:

ROOT

```
|   dir1
|   file1
|   file2
|   file3
|   dir2
|   dir3
|   file1
```

file1

file2

这个图说明: ROOT目录包括三个子目录和两个文件。第一个子目录包含3个文件, 第二个子目录是空的, 第三个子目录有

Input

你的任务是写一个程序读取一些测试数据。每组测试数据表示一个计算机的文件结构。每组测试数据以'*'结尾, 而所有

Output

在显示一个目录中内容的时候, 先显示其中的子目录 (如果有的话), 然后再显示文件 (如果有的话)。文件要求按照名字
你需要注意的是, 我们使用一个'|'和5个空格来表示出缩排的层次。

Sample Input

file1

file2

dir3

dir2

file1

file2

]

]

file4

dir1

]

file3

*

file2

file1

*

#

Sample Output

DATA SET 1:

ROOT

```
|   dir3
|   |   dir2
```

```
|      |      file1
|      |      file2
|      dir1
```

file1

file2

file3

file4

DATA SET 2:

ROOT

file1

file2

Hint

一个目录和它的子目录处于不同的层次

一个目录和它的里面的文件处于同一层次

这个思路非常巧妙，构建了一个树

dir1

fileA

]

fileB

*

ROOT (files: ["fileB"], dirs: [dir_dir1])

↓

dir1 (files: ["fileA"], dirs: [])

```

from sys import exit
class dir:
    def __init__(self, dname):
        self.name = dname
        self.dirs = []
        self.files = []
    def getGraph(self):
        g = [self.name]
        for d in self.dirs:
            subg = d.getGraph()
            g.extend(["| " + s for s in subg])
        for f in sorted(self.files):
            g.append(f)
        return g
n = 0
while True:
    n += 1
    stack = [dir("ROOT")]
    while (s := input()) != "*":
        if s == "#": exit(0)
        if s[0] == 'f':
            stack[-1].files.append(s)
        elif s[0] == 'd':
            stack.append(dir(s))
            stack[-2].dirs.append(stack[-1])
        else:
            stack.pop()
    print(f"DATA SET {n}:")
    print(*stack[0].getGraph(), sep='\n')
    print()

```

230 Kth Smallest Element in a BST

Given the root of a binary search tree, and an integer k, return the kth smallest value

Example 1:

Input: root = [3,1,4,null,2], k = 1

Output: 1

Example 2:

Input: root = [5,3,6,2,4,null,null,1], k = 3

Output: 3

```

class Solution:
    def kthSmallest(self, root, k):
        self.k = k
        self.result = None
        def inorder(node):
            if not node or self.result is not None:
                return
            inorder(node.left)
            self.k -= 1
            if self.k == 0:
                self.result = node.val
                return
            inorder(node.right)
        inorder(root)
        return self.result

```

142 Linked List Cycle II

Given the head of a linked list, return the node where the cycle begins. If there is no cycle, return `None`.
 There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer.
 Do not modify the linked list.

Example 1:

Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node with value 2.

Example 2:

Input: head = [1,2], pos = 0

Output: tail connects to node index 0

Explanation: There is a cycle in the linked list, where tail connects to the first node with value 1.

Example 3:

Input: head = [1], pos = -1

Output: no cycle

Explanation: There is no cycle in the linked list.

循环链表可以用快慢指针去找圈


```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None
class Solution:
    def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                slow = head
                while slow != fast:
                    slow = slow.next
                    fast = fast.next
                return slow
        return None

```

105 Construct Binary Tree from Preorder and Inorder Traversal

Given two integer arrays preorder and inorder where preorder is the preorder traversal

Example 1:

Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]

Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]

Output: [-1]

不要重复复制切片，在函数形参中用指针描述范围即可

树的题一定要学会递归（有点像数学归纳法hhh）

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        hashtable = {key:index for index,key in enumerate(inorder)}
        n = len(preorder)
        def tree(i1,i2,j1,j2,hashtable):
            if i2 < i1:
                return None
            root = TreeNode(preorder[i1])
            k = hashtable[preorder[i1]]
            root.left = tree(i1+1,k-j1+i1,j1,k-1,hashtable)
            root.right = tree(k-j1+i1+1,i2,k+1,j2,hashtable)
            return root
        return tree(0,n-1,0,n-1,hashtable)

```

11 Container With Most Water

You are given an integer array height of length n. There are n vertical lines drawn such that the two lines at indices i and j form a container, such that the container contains the maximum amount of water. Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:

Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the maximum amount of water is 49 units.

Example 2:

Input: height = [1,1]

Output: 1

双指针移动的时候必须局部最优

```

class Solution:
    def maxArea(self, height: List[int]) -> int:
        maxwater = 0
        left, right = 0, (len(height)-1)
        while left < right:
            if height[right] <= height[left]:
                maxwater = max(maxwater, height[right]*(right-left))
                right -= 1
            else:
                maxwater = max(maxwater, height[left]*(right-left))
                left += 1
        return maxwater

```

24591:中序表达式转后序表达式

总Time Limit: 1000ms Memory Limit: 65536kB

Description

中序表达式是运算符放在两个数中间的表达式。乘、除运算优先级高于加减。可以用"()"来提升优先级 --- 就是小学

- 1) 一个数是一个中序表达式。该表达式的值就是数的值。
 - 2) 若a是中序表达式，则"(a)"也是中序表达式(引号不算)，值为a的值。
 - 3) 若a,b是中序表达式，c是运算符，则"acb"是中序表达式。"acb"的值是对a和b做c运算的结果，且a是左操作数，
- 输入一个中序表达式，要求转换成一个后序表达式输出。

Input

第一行是整数n(n<100)。接下来n行，每行一个中序表达式，数和运算符之间没有空格，长度不超过700。

Output

对每个中序表达式，输出转成后序表达式后的结果。后序表达式的数之间、数和运算符之间用一个空格分开。

Sample Input

```

3
7+8.3
3+4.5*(7+2)
(3)*((3+4)*(2+3.5)/(4+5))

```

Sample Output

```

7 8.3 +
3 4.5 7 2 + * +
3 3 4 + 2 3.5 + * 4 5 + / *

```

中缀转后缀算法

数字 扔进输出。

左括号 入栈。

操作符 把栈里比自己同等级或强的都弹出去，然后自己入栈。

右括号 栈内“大清扫”直到遇见左括号。
最后 把栈里剩下的所有东西全部倒出来。

```

import sys
input = sys.stdin.read()
data = input.split()
k = int(data[0])
i = 1
for _ in range(k):
    mystr = data[i]
    num = ""
    output = []
    flagshu = True
    stack = []
    for s in mystr:
        if s.isdigit():
            flagshu = True
            num += s
        elif s == ".":
            flagshu = True
            num += s
        elif s == "*" or s == "/":
            if flagshu:
                flagshu = False
                if num:
                    output.append(num)
                    num = ''
            while stack:
                a = stack.pop()
                if a == "(" or a == "+" or a == "-":
                    stack.append(a)
                    stack.append(s)
                    break
            else:
                output.append(a)
        if not stack:
            stack.append(s)
        elif s == "(":
            if flagshu:
                flagshu = False
                if num:
                    output.append(num)
                    num = ''
            stack.append(s)
        elif s == ")":
            if flagshu:

```

```

        flagshu = False
    if num:
        output.append(num)
        num = ''
    while True:
        a = stack.pop()
        if a == "(":
            break
        output.append(a)
    elif s == "+" or s == "-":
        if flagshu:
            flagshu = False
            if num:
                output.append(num)
                num = ''
            while stack:
                a = stack.pop()
                if a == "(":
                    stack.append(a)
                    stack.append(s)
                    break
                else:
                    output.append(a)
            if not stack:
                stack.append(s)
    if num:
        output.append(num)
    while stack:
        output.append(stack.pop())
    print(*output)
    i += 1

```

394 Decode String

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is repeated k times.

You may assume that the input string is always valid; there are no extra white spaces, and the input string is guaranteed to be a valid input.

The test cases are generated so that the length of the output will never exceed 10^5 .

Example 1:

Input: `s = "3[a]2[bc]"`

Output: `"aaabcbc"`

Example 2:

Input: `s = "3[a2[c]]"`

Output: `"accaccacc"`

Example 3:

Input: `s = "2[abc]3[cd]ef"`

Output: `"abcbcccdcdcdcd"`

Constraints:

$1 \leq s.length \leq 30$

`s` consists of lowercase English letters, digits, and square brackets `'[]'`.

`s` is guaranteed to be a valid input.

All the integers in `s` are in the range `[1, 300]`.

class Solution:

```
def decodeString(self, s: str) -> str:
    s = '1[' + s + ']'
    stack = []
    coeff = 0
    curr_s = ''
    res = ''
    for char in s:
        if char.isdigit():
            coeff = 10 * coeff + int(char)
        elif char == '[':
            stack.append((coeff, curr_s))
            coeff = 0
            curr_s = ''
        elif char != ']':
            curr_s += char
        else:
            p_coeff, prev_s = stack.pop()
            curr_s = prev_s + p_coeff*curr_s
    return curr_s
```

```

class Solution:
    def decodeString(self, s: str) -> str:
        def deductbracket(s):
            mystr = ""
            count = 0
            j1 = float("inf")
            j2 = -1
            for i in range(len(s)):
                if s[i].isalpha() and i < j1:
                    mystr += s[i]
                elif s[i] == "[":
                    count += 1
                    if count == 1:
                        j1 = i
                elif s[i] == "]":
                    count -= 1
                    if not count:
                        j2 = i
                        k = j1 - 1
                        while k >= 0 and s[k].isdigit():
                            k -= 1
                        mystr += (deductbracket(s[j1+1:j2])) * int(s[k+1:j1])
                        j1 = float("inf")
            return mystr
        return deductbracket(s)

```


148 Sort List

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
def sortList(head):
    if not head or not head.next:
        return head
    # 1. 获取链表总长度
    h, length = head, 0
    while h:
        h, length = h.next, length + 1
    dummy = ListNode(0)
    dummy.next = head
    # 2. 自底向上归并
    step = 1
    while step < length:
        curr, tail = dummy.next, dummy
        while curr:
            # 找到左链表 left
            left = curr
            right = split(left, step) # 切断并返回右侧起点
            curr = split(right, step) # 再切断并返回下一组起点
            # 合并有序链表, 并连接到 tail 后面
            tail.next = merge(left, right)
            # 移动 tail 到当前已排序部分的末尾
            while tail.next:
                tail = tail.next
        step *= 2
    return dummy.next
def split(head, n):
    """切断链表, 返回第 n 个节点后的那个节点"""
    for i in range(n - 1):
        if head: head = head.next
    if not head: return None
    next_node = head.next
    head.next = None # 断开连接
    return next_node
def merge(l1, l2):
    """标准的双有序链表合并"""
    dummy = ListNode(0)
    curr = dummy
```

```

while l1 and l2:
    if l1.val < l2.val:
        curr.next, l1 = l1, l1.next
    else:
        curr.next, l2 = l2, l2.next
    curr = curr.next
curr.next = l1 if l1 else l2
return dummy.next

```

06648:Sequence

总Time Limit: 3000ms Memory Limit: 65536kB

Description

给定m个数字序列，每个序列包含n个非负整数。我们从每一个序列中选取一个数字组成一个新的序列，显然一共可以构造

Input

输入的第一行是一个整数T，表示测试用例的数量，接下来是T个测试用例的输入

每个测试用例输入的第一行是两个正整数m ($0 < m \leq 100$) 和n ($0 < n \leq 2000$)，然后有m行，每行有n个数，数字序列中的数字不会大于10000

Output

对每组测试用例，输出一行用空格隔开的数，表示最小的n个和

Sample Input

```

1
2 3
1 2 3
2 2 3

```

Sample Output

```

3 3 4

```

注意空间限制，不能一开始全部处理数据，要一点点读入。

bfs是对的，但是对所有数列bfs会爆空间，两个两个处理很精妙

```

import sys
import heapq
def merge(arr1, arr2, n):
    """
    将两个有序数组 arr1 和 arr2 合并，求出所有组合中最小的 n 个和
    使用堆来进行合并搜索
    """
    heap = []
    visited = set()
    # 初始候选项: (arr1[0]+arr2[0], 0, 0)
    heapq.heappush(heap, (arr1[0] + arr2[0], 0, 0))
    visited.add((0, 0))
    result = []
    while len(result) < n:
        s, i, j = heapq.heappop(heap)
        result.append(s)
        # 如果 arr1 中的下一个数存在，尝试加入候选项
        if i + 1 < n and (i + 1, j) not in visited:
            heapq.heappush(heap, (arr1[i + 1] + arr2[j], i + 1, j))
            visited.add((i + 1, j))
        # 如果 arr2 中的下一个数存在，尝试加入候选项
        if j + 1 < n and (i, j + 1) not in visited:
            heapq.heappush(heap, (arr1[i] + arr2[j + 1], i, j + 1))
            visited.add((i, j + 1))
    return result

def main():
    input_data = sys.stdin.read().split()
    it = iter(input_data)
    T = int(next(it))
    results = []
    for _ in range(T):
        m = int(next(it))
        n = int(next(it))
        # 读取第一个序列，并排序
        current = sorted(int(next(it)) for _ in range(n))
        # 依次与后续的 m-1 个序列合并
        for _ in range(m - 1):
            seq = sorted(int(next(it)) for _ in range(n))
            current = merge(current, seq, n)
        results.append(" ".join(map(str, current)))
    sys.stdout.write("\n".join(results))

```

```
if __name__ == "__main__":  
    main()
```

27300:模型整理

总Time Limit: 1000ms Memory Limit: 65536kB

Description

深度学习模型（尤其是大模型）是近两年计算机学术和业界热门的研究方向。每个模型可以用“模型名称-参数量”命名。例如，Bert-110M, Bert-340M 分别代表参数量为 1.1 亿和 3.4 亿的 Bert 模型，GPT3-350M, GPT3-1.3B 参数量的数字部分取值在 [1, 1000) 区间（一个 8 亿参数的模型表示为 800M 而非 0.8B，10 亿参数的模型表示为 1000M）。计算机专业的学生小 A 从网上收集了一份模型的列表，他需要将它们按照名称归类排序，并且同一个模型的参数量从小到大排序。

Input

第一行为一个正整数 n ($n \leq 1000$)，表示有 n 个待整理的模型。

接下来 n 行，每行一个“模型名称-参数量”的字符串。模型名称是字母和数字的混合。

Output

每行一个“模型名称: 参数量1, 参数量2, ...”的字符串，符号均为英文符号，模型名称按字典序排列，参数量按从小到大排序。

Sample Input

```
5  
GPT-1.3B  
Bert-340M  
GPT-350M  
Bert-110M  
GPT-175B
```

Sample Output

```
Bert: 110M, 340M  
GPT: 350M, 1.3B, 175B
```

Hint

tags: string, sort

__lt__ 是 "less than" 的缩写，对应运算符 <。

当你写 `obj1 < obj2` 时，Python 会自动调用 `obj1.__lt__(obj2)`。

```

from collections import defaultdict
class llm:
    def __init__(self,fullname:str) -> None:
        self.name,self.m=fullname.split('-')
        self.unit=self.m[-1]
        self.num=eval(self.m[:-1])
    def __lt__(self,other):
        if self.unit!=other.unit:
            return self.unit=='M'
        else:
            return self.num<other.num
d = defaultdict(list)
n = int(input())
for i in range(n):
    l = llm(input())
    d[l.name].append(l)
names = sorted(d.keys())
for i in names:
    llms = sorted(d[i])
    print(f'{i}: {" ".join([j.m for j in llms])}')

```